QALIPSIS - User guide

Version 0.4.16-SNAPSHOT

Table of Contents

INTRODUCTION

Overview

Features

- User-friendly interface
- Automation friendly performance verification
- Developer friendly plug-ins
- Aggregated assertions
- Testing, performance, and geographical distribution
- Live results
- Concurrent load simulations
- Operating system independent

Benefits

- For developers
- For IT operations teams

WHAT'S NEW

QALIPSIS CORE CONCEPTS

Overview

Minions

Scenarios

Steps

Assertions

Nodes

- Head
- Factory

QUICKSTART GUIDE: TESTING AN HTTP SERVER WITH QALIPSIS

Before you get started

What this guide covers

Start an HTTP server

Create a project
Prepare a scenario
- Edit the downloaded scenario skeleton
- Configure the scenario basics
- Add steps to the scenario
- Execute the scenario
- Create an archive of your scenario
- Enable event logging

Next steps

SCENARIO SPECIFICATION

Overview
Configuring a scenario specification
Default scenario configuration
- Number of minions
- Ramping strategies
- Retry policy

Load-injected tree
Singleton step
Data distribution from the singleton step to the load minions
- Loop
- Unicast/forward once
- Broadcast

STEP SPECIFICATIONS

Defining and configuring a step specification
Step context
Common load distribution steps
- Fan-out
- Fan-in/joins
- Error processing steps

Operators
- Timing operators
- Filter operators
- Transformation operators
- Caching operators
- Flow operators
- Combining operators

QALIPSIS CONFIGURATION

Starting QALIPSIS in different modes
- Interactive
- Scenario selection

Runtime configuration
- Command-line arguments
- External property sources
- Injecting values into your scenario

DEPENDENCY INJECTION

Injecting a singleton bean
- Creation of beans
- Injection of beans

Qualifying the bean to inject

Injecting a property value

Injecting iterable containers

Summary

PLUGINS

Introduction

Plugin functionality

Plugin integration

Using QALIPSIS plugins

QALIPSIS available plugins

cassandra plugin
- Overview
- Save step
- Search step

elasticsearch plugin
- Overview
- Poll step
- Save step
- Search step

graphite plugin

influxdb plugin
- Overview
- Poll step
- Save step

- Search step
- jackson plugin
  - Overview
  - csvToMap and csvToObject
  - xmlToObject
  - jsonToMap and jsonToObject
- jms plugin
  - Overview
  - Consume step
  - Produce step
- kafka plugin
  - Overview
  - Consume step
  - Produce step
- mongodb plugin
  - Overview
  - Poll step
  - Save step
  - Search step
- netty plugin
  - Overview
  - HTTP step
  - HTTP WITH step
  - CLOSE HTTP step
  - MQTT publish step
  - MQTT subscribe step
  - TCP step
  - TCP WITH step
  - CLOSE TCP step
  - UDP step
- r2dbc-jasync plugin
  - Overview
  - Poll step
  - Save step
  - Search step
- rabbitmq plugin

Overview
Consume step
Produce step
redis-lettuce plugin
Poll step
Save step
Consume step
Produce step
CONFIGURING AND MONITORING TEST CAMPAIGNS
Overview
Logging in to QALIPSIS
Navigating the QALIPSIS Interface
Change to a differnt tenant
View/Update Your profile
Change your password
Logout
Access Campaign pages
Access Series pages
Access Admin Functions
Creating and updating a series
Configuring and running a campaign
Displaying campaign data
Glossary
FREQUENTLY ASKED QUESTIONS
ARCHITECTURE

Introduction

Overview

QALIPSIS is an enterprise-grade load and performance-testing tool especially designed for distributed systems. It is a step above other load test applications that merely evaluate your system internally.

- QALIPSIS is developer-centric with a low learning curve that makes it easy to use and extremely efficient.
- QALIPSIS is developed in Kotlin to gain the benefits of the Java ecosystem and provide seamless system performance.
- The basic out-of-the box QALIPSIS tool is completely free of charge.

  Commercial-licensed-based plugins, the cloud-based application, and other premium extensions are not included in the basic out-of-the box QALIPSIS tool.
- QALIPSIS is designed to test the performance of distributed and monolithic systems.
- QALIPSIS collects data from operating systems, databases, and monitoring tools.
- QALIPSIS cross-checks system metrics and performs data verification to ensure that the test systems is performing as expected.

Features

User-friendly interface

The QALIPSIS Graphical User Interface (GUI) allows users, depending on their role, to configure and run load tests with capabilities including, but not limited to:

- Start/Stop
- Configure the Load Distribution
- Increase/Decrease the Number of Simulated Users
- View Reports
- Schedule a Test

Automation friendly performance verification

Using the Command Line Interface (CLI) or the Gradle plugin, you can automatically and repeatedly verify the condition and performance of your system.

Developer friendly plug-ins

Downloading a ready-to-use project for your IDE is made easier on QALIPSIS, consequently integrating with popular DevOps tools to test your software and track the impact of recent changes.

Aggregated assertions

Aggregated assertions allow you to verify the overall results on a scenario. You can define thresholds to make a test succeed or fail and immediately get insights about the performance and consistency of your tested system.

Testing, performance, and geographical distribution

- QALIPSIS can test distributed systems.
- QALIPSIS can perform as a distributed system.
- QALIPSIS can simulate load from several geographic locations to run comparisons based on location.

Live results

QALIPSIS events and meters provide extended details of load-test operations, providing users with various options for accessing the results. The results visualizations and dashboards, with details of each test-case execution, provide valuable insight for identifying the bottlenecks in your applications and uncovering the components and conditions of failures.

Concurrent load simulations

QALIPSIS simulates the load from thousands to millions of users (e.g., website visitors, connected IoT devices, etc.) and cross-verifies successive values from different sources including requests, responses, infrastructure metrics, messaging platforms and databases.

Operating system independent

QALIPSIS load and performance testing can be performed from any operating system supporting Java with the same impressive results.

Benefits

QALIPSIS was designed with distributed systems and current developer practices in mind. It provides a testing framework to allow developers to work in a real-world environment, writing and running load tests without a steep learning curve.

For developers

QALIPSIS empowers developers to:

- Choose their own testing tools and preferred programming language for writing load-test scenarios.
- Cross-verify data from different sources: bring responses to requests, infrastructure metrics, middleware events and database records in the same port to compare them and verify that they match your expectations.
- Unlock the full power of the Java ecosystem by making use of any library, protocol or system.
- Reuse pieces of their own software to faster develop scenarios.
- Simulate users, third-party software and IoT devices.
- Simulate real-user and device behavior to make load tests more relevant.
- Generate detailed and relevant results by using distributed tracing markers to follow the complete execution workflow from end to end.
- Customize reports and dashboards.
- Perform tests on development machines, containers and virtual machines with excellent results.
- Integrate their standard DevOps workflow for performance testing before the deployment of health checks on production environments.
- Create a scenario in just a few minutes by enabling utilization of any Java library of choice (such as JUnit, Hamcrest, AssertJ) or pieces of software or proprietary libraries.
- Test non-http technologies using numerous plug-ins, such as databases and messaging platforms.

For IT operations teams

QALIPSIS provides IT Operations teams solutions to:

- Monitor how deployments and other IT operations can affect the availability and overall performance of the system.
- Expose bottlenecks and find correlations between load and resource consumption by unifying the verification of load-test and infrastructure metrics.
- Identify issues before users and customers do by verifying the size and configuration of the system’s infrastructure.
- Examine how the system performs during normal and peak loads by running load tests from different geographic locations at varying times.

- Generate exceptional results on virtual engines, containers and conventional development machines.
- Allow for implementation and integration of verifications in their standard DevOps workflow.

What’s new

QALIPSIS 1.0 is the initial product release.

QALIPSIS 1.0 is designed with the scalability and simplicity required to perform load-tests and performance verification on both distributed and stand-alone systems

With QALIPSIS 1.0:

- You can create scenarios with different branches and running with many minions.
- You can use its many native steps and plugins to fire load on your system and feed QALIPSIS with data to verify load strength.

QALIPSIS 1.0 runs standalone and distributed. A history of events and metrics can be logged or saved in Elasticsearch, TimescaleDB and InfluxDB. Plugins for other tools will be available in future releases.

Future releases of QALIPSIS will include more plugins, more steps for load injection and verifications as well as other improvements.

Feedback on this release, including bugs and suggested enhancements, is welcome and may be sent to the QALIPSIS team at {link to the GitHub issues page}.

QALIPSIS core concepts

Overview

Once you have a minimal understanding of the core concepts, you will be ready to create a QALIPSIS scenario to verify that your software is performing as expected.

The aim of QALIPSIS is to execute actions that inject load into IT systems to monitor and validate their behaviors (latency, failure rates, availability, and data consistency).

The actions are performed by minions that inject load into a system and then run through a preconfigured scenario consisting of steps and assertions. If running QALIPSIS on a distributed system, you can create clusters with Head and Factory nodes to inject higher load.

Before attempting to configure QALIPSIS to run on your system, it is important to understand the core concepts of the tool:

- Minions
- Scenarios
- Steps
- Assertions
- Head
- Factory

Minions

Minions are simulations of website users, IoT devices, or other systems. They each perform a complete scenario.

Any number of minions can act on a scenario. The minion count for a scenario is set during scenario configuration (refer to [Scenario specifications]) or at runtime (refer to QALIPSIS configuration).

Note that a minion count entered at runtime overrides the minion count set during scenario configuration.

Scenarios

A QALIPSIS scenario is the single largest component of a load test. It contains steps with a detailed description of all the operations to accomplish, their dependencies, as well as assertions to ensure everything performs as expected.

A simple scenario configuration includes the following:

1. Call to the scenario function.
   - Mandatory
   - QALIPSIS Signature: `@Scenario`
2. Assign the scenario a name.
   - Mandatory
   - Value: alphanumeric string
   - QALIPSIS Signature: `val myScenario = scenario("scenario name")`
3. Define the total # of minions to be executed.
   - Optional
   - Value: positive number
   - Default = 1; Minimum = 1
   - QALIPSIS Signature: `minionsCount = xxxxx`
4. Define the ramping strategy.
   - Mandatory
   - QALIPSIS Signature: `rampUp {strategy( )}`
5. Define the tree where the load will be injected.
   - Mandatory
   - QALIPSIS Signature: `start()`
6. Call the Gradle Plugin for QALIPSIS.
   - Mandatory

```
plugins {
    application
    kotlin("jvm")
    kotlin("kapt")
    id("com.github.johnrengelman.shadow") version "6.0.0"
    id("com.palantir.docker")
}
```

7. Add steps to your scenario.

Steps

Steps are the central components for working with QALIPSIS and can be developed in Kotlin using the fluent API.

Steps depend on data sources, results of other steps or assertions to execute or use their data.

A step specification is configured such that:

- The step receives the configuration closure as a parameter.
- `.configure` is called after the step parameters are set.

Refer to the Step specifications section of this manual for details.

Each step has a unique configuration depending on the purpose of the step. The step configurations all follow the same logic:

1. Declared as a root or successor step.
2. Provided a name.
3. Provided with step context.
4. Delivers output data.

Steps are configured using core operators:

- Assertions
- Timing
- Creating
- Conditional
- Converting
- Error Handling
- Filtering
- Joining
- Mathematical/Aggregate
- Transforming
- Flow
- User-defined

Refer to the Steps section of this manual for details.

Assertions

Assertions perform cross-checks within the scenario by:

- Providing validation of data persistence, message exchanges, and requests to remote systems.
- Setting and testing durations to verify the system's overall responsiveness.

Nodes

Nodes are distributed instances within a QALIPSIS cluster. QALIPSIS nodes are designated as either a Head or a Factory.

Head

The head node of a QALIPSIS cluster is responsible for orchestration and reporting.

Factory

The factory node of a QALIPSIS cluster hosts the minions that will execute the load tests and assertions.

Quickstart guide: Testing an HTTP server with QALIPSIS

In this quickstart guide, you will learn how to test a very simple echo-like HTTP service using QALIPSIS.

Before you get started

Before starting, you need:

- A general understanding of the [QALIPSIS Core Concepts].
- Java 11 SDK or above installed on your system.
- Docker installed on your system.

What this guide covers

This guide will take you through the following steps for testing a simple HTTP service:

1. Start an HTTP server and a docker environment for the project.
2. Generate a project archive.
3. Configure a scenario skeleton.
4. Add steps to the scenario.
   a. An `HTTP` step to send requests to the server.
   b. A `verify` step that asserts that the server's response is "OK" and to verify the server's responses.
   c. An `HTTP WITH` step that reuses the HTTP connection.
   d. A second `assertion` step to verify the server's responses.
5. Execute the scenario using Gradle and view the report.
6. Create a Java archive of your project so that you can transport it as needed and run it anywhere.
7. Enable event logging to allow you to view errors in the report.

Start an HTTP server

1. Create a file named `docker-compose.yml` in your working directory with the following content:

```yaml
version: '3.5'
services:
  http-punching-ball:
    image: aerisconsulting/http-punching-ball
    command:
      - "--https=true"
      - "--ssl-key=http-server.key"
      - "--ssl-cert=http-server.crt"
    ports:
      - "18080:8080"
      - "18443:8443"
```

2. Start the created docker environment with the command `docker compose up -d`.

Create a project

1. In your browser, open the website Bootstrap QALIPSIS.
2. Verify that your system has the pre-requisites listed on the Bootstrap QALIPSIS page.
3. In the "Name" field, enter "Quickstart".
4. In the "plugins" menu, select `netty`.
5. In the assertions libraries menu, select `Kotest`.
6. Click `Generate`. An archive containing a skeleton of the project is proposed for download.
7. Download the proposed project archive.

[insert graphic/screenshot of the Bootstrap QALIPSIS page]

Prepare a scenario

Edit the downloaded scenario skeleton

1. Open the downloaded project archive in your IDE.
2. Edit the file `src/main/kotlin/Quickstart.kt`.

The scenario skeleton should be displayed:

```kotlin
package quickstart
import io.qalipsis.api.annotations.Scenario
class Quickstart{
    @Scenario
    fun quickstart() {
        scenario("quickstart") {
            minionsCount = 1
            rampUp {
            }
        }
```

```
    .start()
    }
}
```

Configure the scenario basics

Configure the scenario to inject 10,000 minions at a rate of 100 minions every 500 ms.

```
        scenario("quickstart-http") {
            minionsCount = 10000
            rampUp {
                regular(periodMs = 500, minionsCountProLaunch = 100)
            }
        }
        .start()
```

Add steps to the scenario

Add and configure steps within the scenario that will send an HTTP request to the server and verify that the server's response is as expected.

Send an HTTP request

Add an `http` step to the scenario that will send a POST request to the server.

```
        .start()  1
        .netty()  2
        .http {   3
            name = "quickstart-http-post"   4
            connect {   5
                url("http://localhost:18080")
                version = HttpVersion.HTTP_1_1
            }
            request { _,_ ->   6
                    SimpleHttpRequest(HttpMethod.POST, "/echo").body(
                    "Hello World!",
                    HttpHeaderValues.TEXT_PLAIN
                    )
            }
            report {
                reportErrors = true   7
            }
        }
```

1. Specifies the starting point for load injection.
2. Accesses the `netty` namespace.

3 Calls the the `http` operator

4 Gives the `http` step a name.

5 Configures the HTPP connection to the server.

6 Configures the HTTP request (HTTP method, path and body) that will be sent to the server.

7 Enables reporting of errors for the `http` step.

Execute the scenario using Gradle: `./gradlew run`. This allows you to observe requests logged by the HTTP server. However, it does not verify that the server is returning what you expect.

[insert screen shot here of the report]

Verify the server's response

To verify that the server is returning what you expect, add a `verify` step to the scenario, just after the `http` step.

```
        .start()
        .netty()
        .http {   1
            // ...
        }
        .verify { result ->
            assertSoftly {
                result.asClue {
                    it.response shouldNotBe null
                    it.response!!.asClue { response ->
                        response.status shouldBe HttpResponseStatus.OK   2
                        response.body shouldContainOnlyOnce "Hello World!"   3
                        response.body shouldContainOnlyOnce "POST"
                    }
                    it.meters.asClue { meters ->
                        meters.timeToFirstByte!! shouldBeLessThanOrEqualTo Duration.ofSeconds(1)
                        meters.timeToLastByte!! shouldBeLessThanOrEqualTo Duration.ofSeconds(2)
                    }
                }
            }
        }.configure {
            name = "verify-post"  3
        }
```

1 Place holder for the http step (`quickstart-http-post`) configuration parameters.

2 Verifies that the HTTP status of the response is `OK`; the step uses `io.netty.handler.codec.http.HttpResponseStatus` for the verification.

3 Verifies that the body of the response contains the string ("Hello World!") sent in the request.

At this point, if you execute the updated scenario, a log file will display showing the status of each response and confirmation that that server is posting the "Hello World!" string.

Reuse the HTTP connection

You can strengthen the relevance of your test by:

1. Adding an `httpWith` step to the scenario that reuses the original HTTP connection (`quickstart-http-post`).

   The same minions that passed through the first `http` step will pass through the `httpWith` step.

2. Adding a second `verify` step to assert that the response from the server is as expected.

```
...
            configure {
                name = "verify-post"
            }
            .netty()   1
            .httpWith("quickstart-http-post") {   2
                report {
                    reportErrors = true
                }
                request { _, _ ->
                    SimpleHttpRequest(HttpMethod.PATCH, "/echo").body(
                        "Hello World!",
                        HttpHeaderValues.TEXT_PLAIN
                    )
                }
            }.verify { result ->
                assertSoftly {
                    result.asClue {
                        it.response shouldNotBe null
                        it.response!!.asClue { response ->
                            response.status shouldBe HttpResponseStatus.OK   3
                            response.body shouldContainOnlyOnce "Hello World!"   4
                            response.body shouldContainOnlyOnce "PATCH"   5
                        }
                        it.meters.asClue { meters ->
                            meters.timeToFirstByte!! shouldBeLessThanOrEqualTo Duration.ofMillis(100)
                            meters.timeToLastByte!! shouldBeLessThanOrEqualTo Duration.ofMillis(200)
                        }
                    }
                }
            }.configure {
                name = "verify-patch"
            }
```

1. Accesses the `netty` namespace.
2. Calls the `httpWith` operator and specifies the previous `http` connection to reuse ("quickstart-http-post").

Note that there is no need to specify parameters for the`httpWith` connection since it reuses the parameters from the previous `http` connection.

3. Verifies that the HTTP status of the response is `OK`; the step uses `io.netty.handler.codec.http.HttpResponseStatus` for the verification.
4. Verifies that the body of the response contains the string ("Hello World!") sent in the request.
5. Not sure I understand what this line of code does…

Execute the scenario

Execute your updated scenario as a Gradle project.

A couple of seconds after execution, a report (similar to the one below) will display on your console.

```
============================================================
=====================  CAMPAIGN REPORT =====================
============================================================

Campaign..........................quickstart-http-4fb6244cd3
Start.............................2022-03-30T16:41:02.597973Z
End...............................2022-03-30T16:41:53.411602Z
Duration..........................50 seconds
Configured minions................10000
Completed minions.................10000
Successful steps executions........39832
Failed steps executions...........42
Status............................FAILED



=====================  SCENARIO REPORT =====================
Scenario..........................quickstart-http
Start.............................2022-03-30T16:41:02.597973Z
End...............................2022-03-30T16:41:53.411602Z
Duration..........................50 seconds
Configured minions................10000
Completed minions.................10000
Successful steps executions........39832
Failed steps executions...........42
Status............................FAILED
Messages:
- ERROR:  step 'quickstart-http-post' - "Success: 9958, Execution errors: 42"
- INFO:   step 'verify-post' - "Success: 9958, Failures (verification errors): 0, Errors (execution
errors): 0"
- INFO:   step '_622niod959' - "Success: 9958, Execution errors: 0"
```

```
- ERROR:  step 'verify-patch' - "Success: 9926, Failures (verification errors): 32, Errors
(execution errors): 0"
```

- Note that the information for the whole campaign as well as for each scenario is displayed.
- In the example above, there are errors, which explains why the execution failed.
- To understand the errors, refer to Enable Event Logging .

Create an archive of your scenario

Execute the statement `./gradlew assemble`.

A JAR archive ending with `-qalipsis.jar` is created in the folder `build/libs`. This archive contains all dependencies and is self-sufficient to execute your scenario on any machine with a Java Runtime Environment installed.

To execute your scenario: . Open a terminal and navigate to the folder where the JAR archive is stored. . Move the JAR archive out of the `build` folder; this will avoid accidental deletion of the archive. . Execute the command `java -jar quickstart-<your-version>-qalipsis.jar`.

Enable event logging

To know the details of the failures In current directory, create a folder called `config`. In this new folder, create a file called `qalipsis.yml` with the following content:

```yaml
events:
  root: warn
  export:
    slf4j:
      enabled: true
```

This enables the logging of the events with severity WARN and higher to a file called `qalipsis-events.log`.

Execute your scenario again; the events file is created and contains the details of the individual failed assertions:

```
2022-03-31T06:33:55.937Z  WARN ---
step.assertion.failure;io.qalipsis.plugins.netty.tcp.ConnectionAndRequestResult@1e869276
MetersImpl(timeToSuccessfulConnect=null, timeToFailedConnect=null, timeToSuccessfulTlsConnect=null,
timeToFailedTlsConnect=null, bytesCountToSend=85, sentBytes=85, timeToFirstByte=PT0.10696475S,
timeToLastByte=PT0.1163375S, receivedBytes=394)
PT0.10696475S should be <= PT0.1S;campaign=quickstart-campaign-4d39026fd7,minion=quickstart-http-
62qr1n87xx,scenario=quickstart,dag=dag-1,step=verify-patch,context-creation=2022-03-
31T06:33:55.730Z,parent-step=_629qthims5,_ step-type=Verification,iteration=0,attempts-after-
```

```
failure=0,isExhausted=true,isTail=true,isCompleted=true_
2022-03-31T06:33:56.916Z  WARN ---
step.assertion.failure;io.qalipsis.plugins.netty.tcp.ConnectionAndRequestResult@19a03b37
MetersImpl(timeToSuccessfulConnect=null, timeToFailedConnect=null, timeToSuccessfulTlsConnect=null,
timeToFailedTlsConnect=null, bytesCountToSend=85, sentBytes=85, timeToFirstByte=PT0.128673083S,
timeToLastByte=PT0.128737375S, receivedBytes=394)
PT0.128673083S should be <= PT0.1S;campaign=quickstart-campaign-4d39026fd7,minion=quickstart-http-
62sqemkv8j,scenario=quickstart,dag=dag-1,step=verify-patch,context-creation=2022-03-
31T06:33:56.735Z,parent-step=_629qthims5,_ step-type=Verification,iteration=0,attempts-after-
failure=0,isExhausted=true,isTail=true,isCompleted=true_
```

Essentially, you want to view the following pertinent parts of the message:

```
2022-03-31T06:33:55.937Z ... PT0.10696475S should be <= PT0.1S ... campaign=quickstart-campaign-
4d39026fd7,... scenario=quickstart,... step=verify-patch
```

- The exact instant when the failed assertion occurred:

  `2022-03-31T06:33:55` (March 31, 2022 at 06:33:55 UTC) in the example above.

- The reason for the failure:

  `PT0.10696475S should be ⇐ PT0.1S`

- The campaign (`quickstart-campaign-4d39026fd7`), scenario (`quickstart`-http), and verification step (verify-patch) where the failed assertion occurred.

Next steps

- Discover how to push events to a database, making them easier to view and analyze.
- Work through other QALIPSIS Quickstart Guides:
  - quickstart 1
  - quickstart 2

Scenario specification

Overview

A scenario specification is the total of all of the steps required to configure the scenario and to execute it at run-time.

In order to provide maximum flexibility for configuring a scenario and the best performance while executing it, QALIPSIS splits its configuration from its execution.

- In the configuration phase, you develop specifications for the scenario.
- In the execution phase, you set the runtime parameters for the scenario(s) you are executing.

This section focuses on scenario configuration. Details on scenario execution are provided in QALIPSIS configuration.

Configuring a scenario specification

A scenario specification is made up of several parts.

1. A call to the Scenario function:

```
@Scenario
fun myScenario()
```

2. Assigning the scenario a name (*hello world*):

```
@Scenario
fun myScenario(){
    val myScenario = scenario("hello-world")
```

3. Configuring the scenario.

   a. Assume the default configuration parameters:

```
@Scenario
fun myScenario()
    val myScenario = scenario("hello-world"){// Scenario default configuration parameter values
    }
```

   The default parameters include:

   - minionCount = 1

- retryPolicy = no retry ---

4. Include defined parameters within the configuration, as in the following example:

```
@Scenario
fun myScenario(){
    val myScenario = scenario("hello-world"){
        minionsCount = 1000
        retryPolicy(instanceOfRetryPolicy)
        rampUp {
        regular(1000, 50)
        }
    }
}
```

5. Identifying the root of the (load-injected) tree receiving all the minions.

```
@Scenario
fun myScenario(){
    val myScenario = scenario("hello-world"){
        minionsCount = 1000
        retryPolicy(instanceOfRetryPolicy)
        rampUp {
        regular(1000, 50)
    }
}
.start()\\Root of the load injected tree.
```

6. Adding step specifications to the tree:

```
@Scenario
fun myScenario(){
    val myScenario = scenario("hello-world"){
        minionsCount = 1000
        retryPolicy(instanceOfRetryPolicy)
        rampUp {
            regular(1000, 50)
        }
    }
}
.start() // Add step specifications here.
```

Default scenario configuration

Every scenario has a default configuration.

Number of minions

- Optional

- Default = 1

Whether you want to simulate users, IoT devices or third-party systems, each simulation is identified with a unique minion:

- The more minions, the more load.
- Minions execute concurrently and through all trees.
- You can specify a default `minionsCount` (1000 in the following example):

```
@Scenario
fun myScenario() {
    val myScenario = scenario("hello-world") {
        minionsCount = 1000
    }
}
```

Load-injected trees receive all of the minions; non-load injected trees receive only one minion.

Ramping strategies

- Mandatory
- There is no default

Once you have specified the number of minions to be injected into your scenario, you need to specify how they will be injected.

You can use an out-of-the-box strategy or create your own.

In the following sections we will explain how to use a few of the out-of-the-box ramping strategies.

- Accelerated Ramping Strategies

  An accelerating ramping strategy injects either a set number of minions at an accelerated pace or an accelerated number of minions at a set pace.

  1. Accelerated pace using `faster`.

     The following example injects 10 minions to start,then 10 new minions after 2000 ms, 10 new minions after another 1333 (2000 / 1.5) ms, 10 new minions after 888 (2000 / 1.5 / 1.5) ms, continuing until the interval reaches 200 ms.

     After the interval reaches 200ms, 10 minions are injected every 200 ms until all minions are active.

     ```
     @Scenario
     fun myScenario() {
     ```

```
    val myScenario = scenario("hello-world") {
        rampUp {
            faster(2000, 1.5, 200, 10)
        }
    }
}
```

The speed-factor at runtime will be applied to the multiplication factor: a speed factor of 2 will reduce the interval time in half.

2. Accelerated number of minions using `more`.

   The following example injects minions every 200 ms: first 10, then 20 new minions after 200 ms, 40 new minions after another 200 ms, to the limit of 1000 minions every 200ms until all minions are active.

```
@Scenario
fun myScenario() {
    val myScenario = scenario("hello-world") {
        rampUp {
            more(200, 10, 2.0, 1000)
        }
    }
}
```

   The speed-factor at runtime will be applied to the # of minions injected: a speed factor of 2 will increase the number of minions by 2 at each injection.

- `regular` Ramping Strategy

  A regular ramping strategy sets the injection of a set amount of minions at a set pace.

  The following example sets the execution of 200 minions every 10 ms.

```
@Scenario
fun myScenario() {
    val myScenario = scenario("hello-world") {
        rampUp {
        regular(200, 10)
        }
    }
}
```

  The speed-factor at runtime will be applied to the interval: a speed factor of 2 will divide the interval by 2.

- `timeframe` Ramping Strategy

  A time frame ramping strategy injects an equal number of minions at an equal interval within a set timeframe.

The following example starts an equal number of minions every 20 ms, so that they all are active after 40000 ms.

```
@Scenario
fun myScenario() {
    val myScenario = scenario("hello-world") {
        rampUp {
            timeframe(20, 40000)
        }
    }
}
```

The speed-factor at runtime will be applied to the interval: a speed factor of 2 will divide the interval by 2.

- User Defined Ramping Strategies

  A user-defined ramping strategy is totally open to the developer's creativity or needs. It consists of returning a `MinionsStartingLine` considering the previous period in milliseconds, the total number of minions to start and the speed factor to apply.

  The following example releases 1/100th of all the minions after 1000 ms, then applies different rules depending on the past interval:

- If the previous interval was longer than 2 seconds, 100th of all the minions are released after 100ms;
- Otherwise, a random interval is calculated applying the speed factor to reduce it, in order to start 10th of all the minions.

```
@Scenario
fun myScenario() {
    val myScenario = scenario("hello-world") {
        rampUp {
            define { pastPeriodMs, totalMinions, speedFactor ->
                when {
                    pastPeriodMs == 0L -> MinionsStartingLine(totalMinions / 100, 1000)
                    pastPeriodMs > 2000 -> MinionsStartingLine(totalMinions / 10, 100)
                    else -> MinionsStartingLine(totalMinions / 10,
                        ((1000 + pastPeriodMs * Math.random()) /
speedFactor).toLong().coerceAtLeast(200))
                }
            }
        }
    }
}
```

Whatever definition you choose:

- The maximum number of minions launched cannot be more than the `totalmMnions`.

- Generating a negative interval results in an immediate start.
- When calling the first iteration of a user-defined strategy, the initial value of `pastPeriodMs` is 0.

Retry policy

- Optional
- Default = no retry

Execution of steps can fail for a variety of reasons, including, but not limited to, an unreachable remote host or invalid data.

When a step execution fails, it may be retried if a retry policy has been set within the scenario configuration or the step configuration.

Once all attempts are executed per the defined `retryPolicy`, the step execution context is marked as exhausted and only error processing steps are executed. Refer to the Step specifications section for more information on error processing steps.

```
@Scenario
fun myScenario() {
    val myScenario = scenario("hello-world") {
        retryPolicy(instanceOfRetryPolicy)
    }
}
```

Load-injected tree

A scenario is a combination of directed acyclic graphs, joined by steps.

There might be several graphs starting concurrently at the scenario root, but only one graph (the specified load-injected branch) is visited by all the minions injected into the scenario.

Other graphs are considered non-injected or "side-trees" and perform data verification for when the branches rejoin.

`myScenariostart().doSomething()` identifies which graph is the root receiving the load of all the minions at runtime.

`myScenario.doSomething()` identifies a second graph visited by a unique minion and generally consisting of iterative operations such as polling of data sources.

```
@Scenario
fun myScenario() {
    val myScenario = scenario("hello-world") {
```

```
        minionsCount = 1000
        rampUp {
            regular(1000, 50)
        }
    }
    myScenario.start() // Root branch receiving all minions.
        .returns<string>{ "My input" }
    myScenario // Second branch visited by a unique minion.
        .returns<String>{ "My other input" }
}
```

Note that only one use of `.start()` is permitted and only one step after it.

To create two load-injected branches, you can use the `.split` command.

`.split` provides the output of a unique step to several steps by initiating the divergence of two load-injected branches, both running concurrently and receiving all the minions.

In the following example, two branches are diverging: one with the step `.map{}` and a second with `flatten().filterNotNull()`. Both run concurrently after `.split`.

```
@Scenario
fun myScenario() {
    val myScenario = scenario("hello-world"){
    .start()
        .split {
             .map {
             .returns<String>{ "My other input"
             }
            flatten() {
                .filterNotNull()
            .returns<String>{ "My other input" }
            }
        }
    }
}
```

Singleton step

A singleton step is designed to be executed only once by one minion. Its purpose is to gather data to provide to the other minions as they pass through it. The minions can use the data to feed other steps or verify expected results in the tested system.

Examples of a singleton step include:

- A database poller.
- A message consumer.

While the singleton step is executed only once, it provides provides data to all the minions passing through it.

A singleton step does not defer from other steps. It is executed using one minion, which looks like any other minion. Even if the singleton step is created on a load-injected branch, it remains aside and is proxied.

The proxy will be executed by all the minions going through the branch, while the singleton step will only have its own one, only aiming at triggering its operation: polling, reading a file, starting a message consumer, etc.

The singleton step and its proxy are connected by a topic, which actually defines the way the polled/read/ consumed records are provided to the minions.

Data distribution from the singleton step to the load minions

For convenience purpose, QALIPSIS proposes three different modes of distributing records out of a singleton step:

- Loop
- Unicast/Forward Once
- Broadcast

Note: You can develop your own singleton step by creating a plugin for it. Details for creating your own plugins will be provided in future updates to this documentation.

QALIPSIS provides all required statements out-of-the-box.

Loop

When the number of records emitted by the singleton step is finite (such as a file), you might want to loop in the whole set of data to be sure there will always be data for the minions.

> Create and insert singleton loop graphic here

All minions receive all the records. When there is no more data for a minion, it receives the same data again from the beginning and loops on the whole set of records as long as it needs new ones.

Unicast/forward once

Each minion receives the next unused record emitted from the step. Once there are no more records to provide, all the minions remain blocked in the step.

Create and insert singleton-unicast graphic here

In the example, each of the 3 minions receives a record fairly: the first asking is served. However, since the buffer is only 2, the first records are just lost, keeping only a buffer of 2 until the first minion requests a value.

Broadcast

All the minions receive all the records from the beginning. Once there are no more records to provide, all of the minions, having already received all available records, remain blocked in the step.

If the size of the buffer is limited, minions landing in the step for the first time will not receive everything from the beginning, but only from the size of the buffer.

Create and insert broadcast graphic here

In the example, the all 3 minions receive all the records in the correct order. However, since the buffer is only 2, each newly requesting minion can only receive two records in the past at the earliest.

Step specifications

Defining and configuring a step specification

A step specification is defined, depending on the step nature and function, as either:

- The root of a scenario (the first step after .start()).

  or

- The successor of a previous step.

In the example below, `my-tcp` is the root step; `reuse-tcp` is a successor step.

```
scenario("hello-netty-udp-world") {
    minionsCount = 100
    rampUp {
        regular(100, minionsCount) // Start all at once.
    }
    .start()
    .netty()
    .tcp { // This step is the root of the scenario.
        name = "my-tcp"
        connect {
            address("localhost", 3000)
            noDelay = true
        }
        request { tcpRequest1 }
        metrics { connectTime = true }
    }
    .reuseTcp("my-tcp") {
        name = "reuse-tcp" // This step is a successor of the previos step and receives the previos step's
output as its input.
        iterate(20, Duration.ofMillis(100))
        request { tcpRequest2 }
    }
}
```

A step specification is configured such that:

- The step receives the configuration closure {} as a parameter (as in `.tcp` and `.reuse-tcp` in the example above).
- `.configure` is called after the step as in the example below:

```
    reuseTcp("my-tcp") {
        request { tcpRequest2 }
    }.configure {
        name = "reuse-tcp"
```

```
        iterate(20, Duration.ofMillis(100))
        retryPolicy(instanceOfRetryPolicy)
        timeout(2000)
    }
```

Step context

A step context is the mechanism by which:

- Data received from the previous step is transported as input to the next steps. The next steps transform the data and provides it as output. The output is forwarded to the next steps as input.
- Prior errors are transported through a step.

Common load distribution steps

Fan-out

To split a branch and add two or more concurrent steps as successors, execute a `.split`.

In the example below, `Anystep` receives the output of the previous step as input. `Anystep` transforms its input per its step configuration. The transformed data is provided as input to both the *map* and the *flatten* steps. The *map* and the *flatten* steps each do something different with the data.

```
    Anystep { }
        .split {
            map { ... }
                .validate{ ... }
        }
            .flatten()
                .filterNotNull()
```

A graphical representation of the above example is shown below. Note that both branches run concurrently, in no defined execution order.

create and include fan-out distribution graphic here→ more like a flowchart

> If the entity returned by the step providing the data before the `.split` (`Anystep()` in the example) is liable to change, do not change it in one branch, as this would have side-effects on the other branch. Consider creating a deep clone of the record that is to be changed to avoid this issue.

Fan-in/joins

Fan-in/Join functions allow the user to re-join split branches of a scenario or to verify data from different sources.

You can join the branches in one of two ways:

- Use the name of a concurrent step.
- Create a step directly in the join.

The following example uses the name of a concurrent step:

```
    Anystep { }
        .innerJoin<EntityWithKey, EntityWithId>(
            on = { "${it.key}" },
            with = "my-datasource-poll-step", // The step `my-datasource-poll-step` is declared in a
parallel branch (or earlier in the same branch) and provides an output of type `EntityWithId` with a field
`id` being a string in this example.
            having = { it.id } // Closure to extract the key from the right side, which should match the
left side value.
        )
        .map { } // The result is a `Pair<EntityWithKey, EntityWithId>` where string representation of `key`
of the `EntityWithKey` equals to the field `id` of the `EntityWithId`.
```

Records coming from the right side are kept in a cache until one comes from the left side with the same key. Once used, they are evicted from the cache.

When a flow comes from the left side but has no corresponding record yet coming from the right, it waits until the defined timeout on the step. Refer to Defining and configuring a step specification for more details.

The step context after the join inherits from the one coming from the left. Below is the graphical representation:

insert operators-inner-join graphic here

The following example creates a step directly in the join:

```
    Anystep  { }
        .innerJoin<EntityWithKey, EntityWithId>(
            on = { "${it.key}" },
            with = {
                r2dbc().poll{

                }
            },
```

```
            having = { it.id }
        )
        .map {}
```

This method is similar to naming a concurrent step except that a step is created in the join. The receiver of the `with` is the scenario.

Error processing steps

When an error occurs when executing a step, after all the potential retries, the step context being carried from top to bottom is marked as "exhausted".

From that point of the process, only the steps relevant for error processing are executed, the others are simply bypassed.

The steps relevant for error processing include:

- `catchError`: takes the collection of errors as parameter to process them.
- `catchExhaustedContext`: takes the full exhausted step context as parameter, where you can also set the `isExhausted` flag to `false` in order execute next steps.

Operators

QALIPSIS provides default operators to tweak the execution of your scenario, that do not require the addition of plugins to your classpath.

Out-of-the-box QALIPSIS operators perform a variety of functions including data transformation, data filtering, data verification, altering the behavior of the data, etc. Together with user-defined operators the list of operators is infinite.

Some of the commonly used operators are defined in this section.

Timing operators

This section provides an explanation and examples of the `delay`, `pace`, `acceleratingPace`, and `constantPace` timing operators.

- `delay` adds a constant delay before executing the next step, keeping the same execution pace as previous step.

insert example and create and insert delay graphic here

- `pace` ensures that the next step is executed at an expected pace, whatever the pace of the input is. The calculation applies independently to each minion.
- `acceleratingPace` reduces the waiting interval for each iteration; to slow down the pace, use an accelerator lower than 1.

> insert example and create and insert acceleratingPace graphic here

- `constantPace` applies a constant pace to execute the successor steps.

> insert example and create and insert constantPace graphic here

Filter operators

This section provides an explanation and examples of the `filter`, `validate`, and `verify` filter operators.

- `filter` removes the step contexts having an input not matching the filter specification. Filter is a silent equivalent of validate; it does not generate an error and does not mark the step context as exhausted.

  > provide an example and insert `filter(is even)` graphic here

- `validate` validates the input. If errors are returned, the step context is marked as exhausted and can then only be processed by error processing steps.

  > Provide an example and insert `validate(is even)` graphic here

- `verify` validates assertions rather than the input data quality/safety that is validated by `validate`.

  > Provide an example and insert `verify(assetThat(it).isEven())` graphic here

Transformation operators

flatThis section provides an explanation and examples of the `flatMap`, `flatten`, `map`, `mapWithContext`, and `onEach` transformation operators.

- `flatMap` converts a collective input into a single element in the output, with a user-defined strategy.

> provide an example and insert `flatMap` graphic here

- `flatten` converts a collective input into a single element in the output for a known collective type (array, iterable, etc.), keeping each item unchanged.

> Provide an example and insert `flatten` graphic here

- `map` converts each element into another element in the output.
- `mapWithContext` provides the same capability as `map` but with the step context as an additional argument.

> Provide an example of `map` and `mapWithContext` and insert a graphic here

- `onEach` executes one or several statements on each element of the input and forwards them unchanged to the output.

> Provide an example of `onEach` and insert a graphic here

Caching operators

This section provides an explanation and examples of the `shelve` and `unshelve` caching operators.

- `shelve` caches the provided values in a shared cache.

  When using a distributed architecture, make sure the cached value can be serialized by the used implementation of shared state registry. The ID of the minion is automatically added to the key to avoid collision.

  > Provide an example of `shelve` and insert a graphic here

- `unshelve` fetches one or more records from the shared state registry, previously shelved with the provided keys for the same minion.

Provide an example of `unshelve` and insert a graphic here

Flow operators

This section provides an explanation and examples of the `pipe`, `blackHole`, and `stage` flow operators.

- `pipe` simply forwards the input to the output.

Provide an example of `pipe` and insert a graphic here

- `blackHole` consumes the input and provides no output.

  Provide an example of `blackhole` and insert a graphic here

- `stage` groups a linear graph of steps.

  Stages are not mandatory in a scenario, but provide a convenient way to iterate steps. In the example below, after `flatten()` is completed it repeats 5 times starting at the first map:

```
stage("my-stage") {
    map { it * 3 }
    .map { arrayOf(it, it + 100) }
    .flatten()
}.configure {
    iterate(5)
}
```

insert a `stage` graphic here if helpful

Combining operators

The most often used combining operator is `collect`.

`collect` collects inputs across all the minions and releases them as a list within the latest received step context.

This operator's function is the opposite of `flatten`.

insert a collect example and a graphic here if helpful

QALIPSIS configuration

Starting QALIPSIS in different modes

Two modes of QALIPSIS configuration are currently supported:

- Interactive
- Scenario Selection

Interactive

1. Execute QALIPSIS with the option `--prompt` or `-p`. as in the following example:

   ```
   java -jar my-scenario-qalipsis.jar --prompt
   ```

2. When prompted, enter the load factor (# of minions), speed factor, ramping strategy, etc.

Scenario selection

Execute QALIPSIS with the `--scenarios` or `-s` as in the following example:

```
java -jar my-scenario-qalipsis.jar --scenarios "my-first-scenario, ?y-*-s*n?rio"
```

The value is a comma-separated list of an exact value of a scenario, a pattern of the scenario, or both. The pattern may contain wildcards (* or ?) as in the example above.

Runtime configuration

Command-line arguments

This section provides an overview and an example of the command-line arguments that are available at runtime.

Note that all command-line arguments can be provided in a file, as explained in External property sources.

1. Number of Minions by Scenario.

   This command defines how many minions should be used in the load-injected tree. It is not compliant with `campaign.minions-factor`.

   Following is an example of the command to use to set the number of minions:

```
---
campaign.minions-count-per-scenario (#ofMinions)
---
```

where `#ofMinions` is an integer and the default value is the `minionCount` set during configuration of the scenario.

2. `Load Factor`.

   The `load factor` command multiplies the number of minions set in the scenario specification by the provided value. It is not compliant with `campaign.minions-count-per-scenario`.

   - If the value is set between 0 and 1, the `minionCount` is reduced.
   - If the value is set to greater than one (or `double`,`triple`, `quadruple`, etc.) the `minionCount` is increased.
   - The default value is always set to `1`.

     Following is an example of the command to set the load factor:

     `campaign.minions-factor, double, default to 1`

3. `Speed Factor`. The speed factor command is applied on the ramping strategy to accelerate or slow down the pace of the minions released.

   For example:

   `campaign.speed-factor, double, default to 1`

   The value for the speed factor is defined the same as for load factor.

External property sources

QALIPSIS uses Micronaut's runtime underneath. Hence, you can use its mechanism of Externalized Configuration with Property Sources.

You can overwrite the default configuration and add your own properties by creating a file called `qalipsis-config.yml` either in the class path or in the working directory or both.

Both the default configuration and the `qalipsis-config.yml` file can specify the same parameters; however, the `qalipsis-config.yml` has precedence.

Injecting values into your scenario

You can inject values into your scenario or its declaring class using the annotation `@io.qalipsis.api.annotations.Property` as in the following example:

```
class EcommercePurchase(@Property("ecommerce-purchase.load") private val numberOfMinions: Int) {

    @Scenario
    fun fromLoginToCheckout(@Property("ecommerce-purchase.action-pace") actionPaceDuration: Duration) {
        scenario("login-select-and-checkout"){
            minionsCount = numberOfMinions
        }
        .constantPace(actionPaceDuration.toMillis())

    }
}
```

Dependency injection

QALIPSIS relies on the Dependency injection framework from Micronaut and supports the injection of dependencies in the constructors of the scenario declaring classes and methods annotated with `@Scenario`.

> Ensure you have the following dependency in the project where your beans or factories are compiled: `kapt("io.qalipsis:api-processors:0.4.16-SNAPSHOT")`

Injecting a singleton bean

You can declare a singleton bean using the annotation `@jakarta.inject.Singleton` on a type, or use a Micronaut factory.

Creation of beans

```
import jakarta.inject.Singleton
@Singleton // The annotation `@jakarta.inject.Singleton` can be added to a class to automatically create a
bean and declare it as singleton.
class NoRetry : RetryPolicy {
    suspend fun <I, O> execute(context: StepContext<I, O>, executable: suspend (StepContext<I, O>) -> Unit)
{
        executable(context)
    }
}
@Factory
class ExecutorFactory {
    @Singleton // The annotation `@jakarta.inject.Singleton` added to a method declares the returned bean
as a singleton.
    fun simpleExecutor(retryPolicy: RetryPolicy): Executor { // The retry policy is automatically injected
into the factory method.
        return SimpleExecutor(retryPolicy)
    }
}
```

Once beans are created, you can inject them in the constructor or the scenario method:

Injection of beans

```
class EcommercePurchase(private val retryPolicy: RetryPolicy) {// The previously created bean of type
`RetryPolicy` is injected in the constructor of the scenario declaring class.

    @Scenario
    fun fromLoginToCheckout(executor: Executor) {// The previously created bean of type `Executor` is
```

```
injected in the scenario declaring method.
        scenario("login-select-and-checkout"){
            retryPolicy(retryPolicy)
        }
        // ...
    }
}
```

Qualifying the bean to inject

When several beans exist for the same type, you might want to select the one to inject. QALIPSIS supports named injection as in the following example.

```
import jakarta.inject.Named

class EcommercePurchase(@param:Named("default") private val retryPolicy: RetryPolicy) { // When the
constructor parameter is a property, precise the target of the annotation.

    @Scenario
    fun fromLoginToCheckout(@Named("simple") executor: Executor) { // The annotation `@Named`
selects the bean of type `Executor` qualified by the name "simple" for the injection.
        scenario("login-select-and-checkout"){
            retryPolicy(retryPolicy)
        }
        // ...
    }
}
```

QALIPSIS only supports name qualifiers; however, you can read more about other qualifiers in the Micronaut documentation.

Injecting a property value

Properties for the runtime property sources are also injectable in scenario declaring classes, constructors, and methods.

```
import io.qalipsis.api.annotations.Property

class EcommercePurchase {

    @Scenario
    fun fromLoginToCheckout(@Property(name = "ecommerce-purchase.action-pace", orElse = "10s")
actionPaceDuration: Duration) {
        scenario("login-select-and-checkout"){
            // ...
        }
        .constantPace(actionPaceDuration.toMillis())
        // ...
```

```
    }
}
```

The default value set in `orElse` is optional.

More information about property sources is available in the Micronaut documentation.

Injecting iterable containers

QALIPSIS supports the injection of a single instance of beans as shown above, as well as an `Iterable` or any of its subtypes `Collection`, `List`, `Set` and their derivatives and `Optional` for properties as in the example below.

```
class EcommercePurchase(private val retryPolicies: List<RetryPolicy>) {

    @Scenario
    fun fromLoginToCheckout(@Property("startDelay") executor: Optional<Duration>) {

        // ...
    }
}
```

Summary

The sample code below includes all the supported methods of injection for beans and properties.

The example is for a scenario method, but can also be used for the constructor of a class declaring scenarios.

```
    @Scenario
    fun fromLoginToCheckout(
        classToInject: ClassToInject, // An instance of `ClassToInject` is injected, failing if missing.
        mayBeOtherClassToInject: Optional<OtherClassToInject>,  // An instance of
`Optional<OtherClassToInject>` is injected, replaced by an empty optional if missing.
        @Named("myInjectable") namedInterfaceToInject: InterfaceToInject, // An instance of
`InterfaceToInject` qualified by the name `myInjectable` is injected, failing if missing.
        injectables: List<InterfaceToInject>, // A `List` of all the instances of classes inheriting from
`InterfaceToInject` is injected
        @Named("myInjectable") namedInjectables: List<InterfaceToInject>, // A `List` of all the instances
of classes inheriting from `InterfaceToInject` and qualified by the name `myInjectable` is injected
        @Property(name = "this-is-a-test") property: Duration, // The value of the property `this-is-a-
test` is injected after its conversion to a `Duration`, failing if missing
        @Property(name = "this-is-another-test", orElse = "10") propertyWithDefaultValue: Int, // The value
of the property `this-is-another-test` is injected after its conversion to a `Int`, replace by 10 as Int if
missing
        @Property(name = "this-is-yet-another-test") mayBeProperty: Optional<String> // The value of the
property `this-is-yet-another-test` is injected after its conversion to a `String`, replaced by an empty
optional if missing
```

```
    ) {
    }
```

Plugins

Introduction

A plugin is a technical extension designed to extend the core capabilities of QALIPSIS.

Plugins:

- Tackle the complexity implied by the multiplicity of technologies and use cases to support in the tool.
- Reduce the coupling between the technical elements, keeping them safer and easier to maintain.

The main advantages of using plugins are:

- A drastic reduction in the footprint of required libraries to run a scenario.
- A better user/developer experience, including:
  - The ability to focus on what is absolutely relevant to the user/developer.
  - Having a smaller list of proposals when using auto-completion.
  - Faster compilation and start times.
  - Smaller artifacts to store, transport and share.

By nature, a plugin is an optional component of QALIPSIS. It allows you to include only what you need while ignoring components and libraries that do not serve your your present concerns.

While a certain number of plugins are proposed by the community to cover the most common use cases and technologies, a plugin can also be an extension only for yourself and remain private.

Plugin functionality

A plugin contains one or several technical functionalities of the software that are not part of the core. The most common functionalities that a plugin might implement are:

- A step to perform an action, such as:
  - Executing requests on a remote system using a specific protocol.
  - Polling data from a source.
  - Transforming data.
- An events logger.

- A meter registry.

Default implementations of the above functionalities are part of the QALIPSIS core. However, the default implementations may not be suited for your scenario (e.g., if productive testing or constant monitoring are requirements). QALIPSIS plugins enhance the above functionalities by providing related implementations.

Plugin integration

To create a test scenario, a tiny Gradle project is required. The plugin is simply added as a dependency to the class path of the project. You can read more about scenario configuration within the Quickstart guide: Testing an HTTP server with QALIPSIS.

If the development of a scenario requires the plugin to compile (when using a step, for example), the dependency to the plugin artifact must be added to the `implementation` dependencies.

If the plugin delivers functionalities that you only need at runtime, add it to the `runtimeOnly` dependencies.

Using QALIPSIS plugins

QALIPSIS provides several plugins targeting different technologies, systems and protocols.

To use a plugin, add the related dependency to your project:

Gradle - Groovy DSL:

```
implementation group:'io.qalipsis', name: 'plugin-XXX', version: '0.4.16-SNAPSHOT'
```

Gradle - Kotlin DSL:

```
implementation("io.qalipsis:plugin-XXX:0.4.16-SNAPSHOT")
```

QALIPSIS available plugins

In this section, documentation is provided for currently available plugins that can be used with your QALIPSIS project.

This section will be updated as more plugins are available.

Note: Before using a plugin, confirm that the related dependency has been added to your project:

- Gradle - Groovy DSL:

```
implementation group:'io.qalipsis', name: 'plugin-XXX', version: '0.4.16-SNAPSHOT
```

- Gradle - Kotlin DSL:

```
implementation("io.qalipsis:plugin-XXX:0.4.16-SNAPSHOT")
```

Name	Description
cassandra	Connects QALIPSIS to APACHE Cassandra.
elasticsearch	Connects QALIPSIS to Elasticsearch.
graphite	Waiting on info…
influxdb	Connects QALIPSIS to InfluxDB.
jackson	Supports reading a file (JSON, XML or CSV), converting the data to an object, and pushing the object to the next step.
jms	Connects QALIPSIS to a messaging platform that supports JMS (for example, ActiveMQ).
kafka	Connects QALIPSIS to an Apache Kafka cluster.
mongodb	Connects QALIPSIS to MongoDb.
netty	Supports TCP, UDP, HTTP and MQTT clients
r2dbc-jasync	Connects connects QALIPSIS to MariaDB, MySQL and PostgreSQL.
rabbitmq	Connects QALIPSIS to RabbitMQ.
redis-lettuce	Connects QALIPSIS to Redis.

cassandra plugin

Overview

The `cassandra` plugin connects QALIPSIS to APACHE Cassandra.

Technologies addressed

- Cassandra: APACHE Cassandra
- Cassandra Query Language: The Cassandra Query Language (CQL)
- DataStax Driver: DataStax.

Dependency

`io.qalipsis:plugin-cassandra`

Namespace in scenario

`cassandra()`

Client library

DataStax: refer to the DataStax.

Datastax is a modern, feature-rich and highly tunable Java client library for Apache Cassandra using Cassandra's binary protocol and Cassandra Query Language.

Supported steps

- `poll`: polls the newest data periodically.
- `save`: saves records generated from the input and step context.
- `search`: searches records and builds a query from the input and step context.

===Poll step The `poll` step within the `cassandra` plugin polls the newest records from the database table.

Ancestor

Scenario or step

Functionality

The `poll` step is created only once in a scenario; it is then used throughout the scenario as called. The `poll` step within the `cassandra` plugin uses a strategy of "delivered at least once."

Example

```
cassandra().poll {
    connect {
       servers = listOf("localhost:7777", "10.42.42.42:7778")
       keyspace = "test_keyspace"
       datacenterProfile = DriverProfile.LOCAL
       datacenterName = "test_datacenter"
   }
   query("test")
   parameters(listOf(1, "two"))
   tieBreaker {
       name = "name"
       type = GenericType.STRING
   }
    pollDelay(Duration.ofSeconds(1))
    monitoring {
        meters = true
        events = false
   }
   broadcast(123, Duration.ofSeconds(20))
}
```

Notable parameters

- `connect` (required): configures the connection to APACHE Cassandra
  - `keyspace` (required): the outermost object that determines how data replicates on nodes.
  - `datacenterProfile` (required): an execution profile that encapsulates a group of settings associated with individual queries.
- `query` (required): defines the prepared statement to execute when polling.
- `parameters` (optional): binds values to the clauses of the prepared query.
- `tieBreaker` (required): repeats the first column used for sorting and comparison (`⇐` or `>=`, depending on the ascending or descending order).
- `pollDelay` (required): specifies the delay between query executions.
- `monitoring` (optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.

Tips

- Use the `poll` step with a left join operator to verify the values saved by your tested system in the database.
- `query` requires at least one sorting field (`tiebreaker` in the example) to filter the newest results and not fetch the same records repeatedly.
- The latest known value for `tiebreaker` is used to filter out values already received from the database.

- Refer to the APACHE Cassandra documentation further parameter and configuration information.

Save step

The `save` step within the `cassandra` plugin persists records into a cassandra database using the save query with Cassandra Query Language (CQL).

Ancestor

Step

Functionality

The `save` step's input and step context are used to generate documents to save into a table.

Example

```
cassandra().save {
   connect {
      servers = listOf("localhost:7777", "10.42.42.42:7778")
      keyspace = "test_keyspace"
      datacenterProfile = DriverProfile.LOCAL
      datacenterName = "test_datacenter"
   }
    tableName = tableName
    columnsConfig = columns
    rowsFactory = rowsFactory
    monitoring {
        meters = true
        events = false
   }
}
```

Notable parameters

- `connect` (required): configures the connection to APACHE Cassandra.
  - `keyspace` (required): the outermost object that determines how data replicates on nodes.
  - `datacenterProfile` (required): an execution profile that encapsulates a group of settings associated with individual queries.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Use the `save` after a `collect` step in order to reduce the number of queries executed onto the database and reduce the load your scenario generates on it.
- The `save` step is extremely flexible; table, rows, and columns depend on the context and input value received from the previous step.

- Refer to the APACHE Cassandra documentation for further parameter and configuration information.

Search step

The `search` step within the `cassandra` plugin searches records in the database using a search query.

Ancestor

step

Functionality

The `search` step's input and step context are used to complete the search; the list of results is then forwarded to the next step(s).

Example

```
cassandra().search {
connect {
       servers = listOf("localhost:7777", "10.42.42.42:7778")
       keyspace = "test_keyspace"
       datacenterProfile = DriverProfile.LOCAL
       datacenterName = "test_datacenter"
}
   query("test")
   parameters(listOf(1, "two"))
   tieBreaker {
       name = "name"
       type = GenericType.STRING
    }
    monitoring {
        meters = true
        events = false
   }
}
```

Notable parameters

- `connect` (required): configures the connection to APACHE Cassandra.
  - `keyspace` (required): the outermost object that determines how data replicates on nodes.
  - `datacenterProfile` (required): an execution profile that encapsulates a group of settings associated with individual queries.
  - `query` (required): defines the prepared statement to execute when searching.
- `parameters`: (optional): binds values to the clauses of the prepared query.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Use the `search` step after a `collect()` step to reduce the number of queries executed onto the database and to reduce the load your scenario generates on the test system.
- The `query` Parameter may contain ordering clauses.
- Refer to the APACHE Cassandra documentation for further parameter and configuration information.

elasticsearch plugin

Overview

The elasticsearch plugin connects QALIPSIS to Elasticsearch.

Technology addressed

Elasticsearch: https://www.elastic.co/

Dependency

`io.qalipsis:plugin-elasticsearch`

Namespace in scenario

`elasticsearch()`

Client library

JAVA API: refer to Elasticsearch JAVA Client.

Supported steps

- `poll`: polls the newest data periodically.
- `save`: saves a batch records generated from the input and step context.
- `search`: searches records and builds a query from the input and step context.

Poll step

The `poll` step within the `elasticsearch` plugin periodically polls data from Elasticsearch. The output is a list of `ElasticsearchDocument` containing JSON data or deserialized to objects.

Ancestor

Scenario

Functionality

The `poll` step is created only once in a scenario; it is then used throughout the scenario as called. The `poll` step within the elasticsearch plugin uses a strategy of “delivered exactly once”. The output of the `poll` step is an `ElasticsearchDocument` that contains maps of column to values.

Example

```
elasticsearch().poll {
    index("building-moves")
    client {
```

```
        RestClient.builder(HttpHost("localhost", esPort, "http")).build()
    }
    query {
        """{
            "query": {
                "term": {
                    "action": "IN"
            }
        },
        "size": 2,
        "sort":["timestamp","username"]
    }""".trimIndent()
    }
    pollDelay(Duration.ofSeconds(1))
}
```

Notable parameters

- `index`: sets the `elasticsearch` indices to use for the queries; defaults to `all`.
- `client`: configures the REST client to connect to Elasticsearch.
- `query`: configures the builder for the JSON query to perform the first poll.
- `queryParameters` (optional): adds parameters to the query.
- `mapper`: configures the JSON Mapper to deserialize the documents.
- `tieBreaker` (required): repeats the first column used for sorting and comparison (`⇐` or `>=`, depending on the ascending or descending order).
- `pollDelay` (required): specifies the delay between query executions.
- `monitoring` (optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.

Tips

- The `poll` step is generally used in conjunction with a `join` to assert data or inject data into a workflow.
- The `query` parameter requires at least one sorting field (`tiebreaker` in the example) to filter the newest results and not fetch the same records repeatedly.
- The latest known value for `tiebreaker` is used to filter out values already received from the database.
- The output of the `poll` step is an `ElasticsearchDocument` that contains maps of column names to values. If `flatten` is called, the records are provided one by one to the next step; otherwise each poll batch remains complete.
- Refer to Elasticsearch Query Parameters for more information on query parameters.

- Refer to the Elasticsearch documentation for further parameter and configuration information.

Save step

The `save` step within the `elasticsearch` plugin persists a batch of documents into Elasticsearch.

Ancestor

Step

Functionality

The `save` step's input and step context are used to generate a list of documents that are forwarded to Elasticsearch.

Example

```
elasticsearch().save {
    documentsFactory = documentsFactory
    client{
        RestClient.builder(HttpHost("localhost", esPort, "http")).build()
    }
        monitoring {
            events = false
        meters = true
    }
}
```

Notable Parameters

- `documentsFactory`: defines the strategy to create documents to save from the input and step context.
- `client`: configures the REST client to connect to Elasticsearch.
- `mapper`: configures the JSON Mapper to serialize the documents.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tip

Refer to the Elasticsearch documentation for further information on parameter configuration.

Search step

on The `search` step within the `elasticsearch` plugin searches documents in the Elasticsearch database using a search query.

Ancestor

Step

Functionality

The `search` step's input and step context are used to generate a query; the batch of results is then forwarded to the next step(s).

Example

```
elasticsearch().search {
    client(clientBuilder)
    mapper(mapperConfigurer)
    index(indicesFactory)
    query(queryFactory)
    queryParameters(paramsFactory)
    monitoring {
        all()
    }
         fetchAll()
```

Notable parameters

- `client`: configures the REST client to connect to Elasticsearch.
- `query`: configures the builder for the JSON query to perform a search.
- `index`: sets the `elasticsearch` indices to use for the queries; defaults to `all`.
- `queryParameters` (optional): adds parameters to the query.
- `mapper` configures the JSON Mapper to deserialize the documents.
- `monitoring` (optional): configures the monitoring parameters for the step.
- `fetchAll`: specifies that all the documents have to be fetched, even over the result-sized window.

Tips

- `fetchAll` must be combined with either a scroll activation or a sorting definition in the query. Refer to the current; The latter case uses `search` after to page all the data. New data coming between two requests will be fetched. The requests stops when the number of fetched documents reaches the total hits of the latest response. Note that this could lead to an infinite step execution if data are added to the results set in `elasticsearch` on a regular basis.
- Refer to the Elasticsearch documentation for further information on parameter configuration.
- For specifics on `queryParameters` refer to Elasticsearch Query Parameters.
- For specifics on `search` parameters, refer to Elasticsearch Search Parameters

graphite plugin

The `graphite_` plugin connects or supports…

influxdb plugin

Overview

The `influxdb` plugin connects QALIPSIS to InfluxDB.

Technologies addressed

- InfluxDB: https://www.influxdata.com/
- Flux: https://docs.influxdata.com/flux/v0.x/

Dependency

`io.qalipsis:plugin-influxdb`

Namespace in scenario

`influxdb()`

Client library

influxdb-client-kotlin: refer to the influxdb-client-kotlin.

Supported steps

- `poll`: polls the newest data periodically.
- `save`: saves a batch of records generated from the input and step context.
- `search`: searches records and builds a query from the input and step context.

Poll step

The `poll` step within the `influxdb()` plugin polls the newest records from the database, with a delay between each execution of a select query.

Ancestor

Scenario

Functionality

The `poll` step is created only once in a scenario; it is then used throughout the scenario as called. The `poll` step within the `influxdb` plugin uses a strategy of “delivered at least once”.

Example

```
influxdb().poll {
    name = "poll.in"
    connect {
        server("http://localhost:8080", BUCKET, ORGANIZATION)
```

```
        basic("user", "passpasspass")
    }
    query("from(bucket: \"$BUCKET\") |> range(start: 0) |> filter(fn: (r) => (r[\"tag\"] == \"IN\") |>
sort([\"_time\"], desc: false")
    pollDelay(Duration.ofSeconds(1))
}
```

Notable parameters

- `connect` (required): configures the connection to InfluxDB.
- `query` (required): specifies a query respecting the requirements of Flux v.2.0.
- `tieBreaker` (required): repeats the first field used for sorting and comparison (`⇐` or `>=`, depending on the ascending or descending order).
- `parameters` (optional): binds values to the clauses of the prepared query.
- `pollDelay` (required): specifies the delay between query executions.
- `monitoring`(optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.

Tips

- Use the `poll` step with a join operator to verify the values that the tested system saves to the database.
- The `token` can be created automatically using your password and username.
- `query` requires at least one sorting field (`tiebreaker` in the example) to filter the newest results and not fetch the same records repeatedly.
- The latest known value for `tiebreaker` filters out the values already received directly from the database.
- Refer to the InfluxDB documentation for further parameter and configuration information.

Save step

The `save` step within the `influxdb()` plugin persists a batch of records into the `influxdb` bucket using a specific `points` class.

Ancestor

Step

Functionality

The `save` step's input and step context are used to generate the bucket, the list of points with their measurement, and tags and fields, which are then combined and forwarded to the database.

Example

```
influxdb().save {
   connect {
        server(
           url = "http://localhost:8080",
           bucket = "test",
           org = "testtesttest"
           )
         basic(
           user = "user",
           password  = "passpasspass"
           )
    }
    query {
        bucket = bucketName
        organization = orgName
        points = pointSupplier
    }
    monitoring {
        events = true
        meters = false
    }
}
```

Notable parameters

- `connect` (required): configures the connection to InfluxDB.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Use the `save` step after a `collect` step to reduce the number of queries executed onto the database and to reduce the load your scenario generates on the test system.
- The step is extremely flexible: bucket, tags, measurement and fields might depend on the context and input value received from the previous step.
- `token` can be created automatically using your password and username.
- Refer to the InfluxDB documentation for further parameter and configuration information.

Search step

The `search` step within the `influxdb` plugin searches records in the database using a search query.

Ancestor

Step

Functionality

The `search` step's input and step context are used to generate a query; the batch of results is then forwarded to the next step(s).

Example

```
influxdb().search {
connect {
   InfluxDBClientKotlinFactory.create(
       InfluxDBClientOptions.builder()
           .url("http://localhost:8080")
           .authenticate("username", "password".toCharArray())
           .org("organizationName")
           .build()
   )
}
search {
   connectionConfig = { _, _ -> influxDbSearchStepConnectionImpl }
   query = { _, _ -> "query" }
}
monitoring {
   events = true
   meters = false
}
}
```

Notable parameters

- `search`: generates the search query from the step input and context.
- `connect` (required): configures the connection to InfluxDB.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Use the `search` step after a `collect` step to reduce the number of queries executed onto the database and to reduce the load the scenario generates on the test system.
- `token` can be created automatically using your password and username.
- Refer to the InfluxDB documentation for further parameter and configuration information.

jackson plugin

Overview

The `jackson` plugin supports reading a (JSON, XML or CSV) file, converting the data to an object, and pushing the object to the next step.

Technologies addressed

Class JsonMapper

Dependency

`io.qalipsis:plugin-jackson`

Namespace in scenario

`jackson()`

Client library

JsonMapper: refer to FasterXML.

Supported steps

- csvToMap and csvToObject: reads a csv file, and converts the data to a Map (or an Object).
- jsonToMap and jsonToObject: reads a json file and converts the data to a Map (or Object).
- `xmlToObject`: reads an xml file and converts the data to an object.

csvToMap and csvToObject

The `csvToMap` and `csvToObject` steps within the `jackson` plugin read a CSV resource (file, classpath resource, URL) and returns each item as an instance of an Object or Map.

Ancestor

Step

Functionality

The `csvToMap` and `csvToObject` step specification has a `targetClass` property to which the output lines should be mapped. The step reads a CSV file, converts the data to a `map` (or an `object`) and pushes the `map` (or `object`) to the next step.

Example

```
jackson().csvToMap {
   classpath(file)
   header {
```

```
        escapeChar('E')
        quoteChar('"')
        columnSeparator(';')
        withHeader()
        allowComments()

        column("1st value").double()
        column("2th value").string().array(",")
        column("3rd value").string(true)
        column("4th value").string(true)
    }
}
jackson().csvToObject(PojoForTest::class) {
    classpath(file)
}
```

Notable parameters

- `classpath(file)`: reads the data from a specified class path resource; `file` is the path to the resource with the leading slash ignored.
- `file(path)`: reads the data from a specified plain file in the file system; `path` is the path to the file, either absolute or relative to the current working directory.
- `url(url)`: specifies and links to the `url` of the data resource.
- `mapper`: configures the JSON Mapper to deserialize the records.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Only one path (`classpath(file)`, `file(path)`, or `url(url)`) to the resource is allowed for each `csvToObject` or `csvToMap` step.
- Refer to the jackson documentation for further congiguration and parameter information.

xmlToObject

The `xmlToObject` step within the `jackson` plugin reads an XML resource (classpath, file, or url) and returns each item as an instance of an object of the specified class.

Ancestor

Step

Functionality

The `xmlToObject` step specification has a `targetClass` property to which the output lines should be mapped. The step reads an XML file, converts the data to an `Object`, and pushes the `Object` to the next step.

Example

```
jackson().xmlToObject(PojoForTest::class) {
   classpath(file)
}
```

Notable parameters

- `classpath(file)`: reads the data from a specified class path resource; `file` is the path to the resource with the leading slash ignored.
- `file(path)`: reads the data from a specified plain file in the file system; `path` is the path to the file, either absolute or relative to the current working directory.
- `url(url)`: specifies and links to the `url` of the data resource.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Only one path (`classpath(file)`, `file(path)`, or `url(url)`) to the resource is allowed for each `xmlToObject` step.
- Refer to the jackson documentation for further congiguration and parameter information.

jsonToMap and jsonToObject

The `jsonToMap` and `jsonToObject` steps within the `jackson` plugin read a JSON resource (file, classpath resource, URL) and returns each item as an instance of `map` or `object`).

Ancestor

Step

Functionality

The `jsonToMap` and `jsonToObject` step specification has a `targetClass` property to which the output lines should be mapped. The step reads a JSON file, converts the data to a `Map` (or an `Object`) and pushes the `Map` (or `Object`) to the next step.

Example

```
jackson().jsonToMap() {
   classpath(file)
}
jackson().jsonToObject(PojoForTest::class) {
   classpath(file)
}
```

Notable parameters

- `classpath(file)`: reads the data from a specified `path` resource; `file` is the path to the resource with the leading slash ignored.
- `file(path)`: reads the data from a specified plain file in the file system; `path` is the path to the file, either absolute or relative to the current working directory.
- `url(url): specifies and links to the `url` of the data resource.
- `monitoring`(optional): configures the monitoring parameters for the step.

Tips

- Only one path (`classpath(file)`, `file(path)`, or `url(url)`) to the resource is allowed for each `jsnToObject` or `jsnToMap` step.
- Refer to the jackson documentation for further configuration and parameter information.

jms plugin

Overview

The `jms` plugin connects QALIPSIS to a messaging platform that supports JMS.

Technology addressed

JMS: https://www.oracle.com/java/technologies/java-message-service.html

Dependency

`io.qalipsis:plugin-jms`

Namespace in scenario

jms()

Client library

JMS API: refer to Java Interface Summary.

Supported steps

- `consume`: consumes data from a messaging platform supporting JMS.
- `produce`: pushes data to a messaging platform supporting JMS.

Consume step

The `consume` step within the `jms` plugin consumes records from ActiveMQ.

Ancestor

Scenario

Functionality

The `consume` step polls data from the designated ActiveMQ connection, step context, and the data deserializer.

Example

```
.jms()
    .consume {
        queues("queue-1")
        queueConnection { ActiveMQConnectionFactory().createQueueConnection() }
        monitoring {
            events = true
            meters = true
        }
    }.deserialize(JmsJsonDeserializer(User::class))
```

Notable parameters

- `consume`: configures the the ActiveMQ connection.
  - `queues`: specifies the queue(s) for the connection.
  - `QueueConnection`: provides the java ActiveMQ connection for the specified `queues`.
  - `topics`: specifies the topic(s) for the connection.
  - `TopicConnection`: provides the java ActiveMQ connection for the specified `topics`.
- `monitoring` (optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.
- `deserialize`: specifies the class to use to deserialize the body of the response.

Tips

- The ActiveMQ connection can be configured for either `queues` (`QeueConnection`) or `topics` (`TopicConnection`), but not both.
- Refer to the JMS documentation for for further parameter and configuration information.

Produce step

The `produce` step within the `jms` plugin produces records in ActiveMQ using JMS.

Ancestor

Step

Functionality

The `produce` step produces a batch of records into one or more JMS `destinations`. The record values may be provided by previous steps.

Example

```
.jms().produce {
    records { ctx, input ->
        listOf(
            JmsProducerRecord(
                destination = ActiveMQQueue().createDestination("queue-1"),
                messageType = JmsMessageType.TEXT,
                value = "hello-queue"
            ),
            JmsProducerRecord(
                destination = ActiveMQQueue().createDestination("queue-1"),
                messageType = JmsMessageType.AUTO,
                value = "another message".toByteArray()
            )
        )
    }
    connect { ActiveMQConnectionFactory().createQueueConnection() }
```

```
        monitoring {
            events = true
            meters = true
        }
}
```

Notable parameters

- `records`: generates and configures a list of produced records.
  - `destination`: defines the destination for each `JmsProducerRecord` as either queues (`QueueConnection`) or topics (`TopicConnection`).
  - `messageType`: indicates the `JmsMessageType` as: `TEXT`, `AUTO`, `BYTES`, or `OBJECT`.
  - `value`: defines the message to be displayed at the `destination`.
- `connect`: specifies the connection type for the `JmsProducerRecord`(s).
- `monitoring`(optional): configures the monitoring parameters for the step.

Tips

- If you are not sure about the `JmsMessageType`, use `AUTO` to allow the system to infer the type and display the result.
- Refer to the JMS documentation for further parameter and configuration information.

kafka plugin

Overview

The `kafka` plugin connects QALIPSIS to an APACHE Kafka cluster.

Technology addressed

Kafka: https://kafka.apache.org/

Dependency

`io.qalipsis:plugin-kafka`

Namespace in scenario

`kafka()`

Client library

Kafka: refer to Apache Kafka

Supported steps

- `consume`: consumes data from topics of an Apache Kafka cluster.
- `produce`: pushes data to topics of the Apache Kafka cluster.

Consume step

The `consume` step within the `kafka` plugin polls data from topics of an Apache Kafka cluster and forwards the data to the next step, either in batches or individually.

This step is generally used in conjunction with a left join to assert data or inject data into a workflow.

The `consume` step supports multithreading (concurrent consumers) using the `concurrency` property.

Ancestor

Scenario

Functionality

The `consume` step starts a polling loop for the designated `client`, using the provided subscription `topics` name, or `topicsPattern` and `groupId`. When new records appear, `consume` polls them and sends them to the next step(s).

Example

```
kafka().consume {
   name = "kafka-consumer"
   bootstrap(bootstrap)
   topics(testProducerTopic)
   groupId("kafka-left")
   pollTimeout(100)
   offsetReset(OffsetResetStrategy.EARLIEST)
}
```

Notable parameters

- `bootstrap`: configures the bootstrap hosts of the `Kafka` cluster; defaults to localhost:9092.
- `groupId`: defines the consumer group.
- `pollTimeout` defines the consumer poll timeout to wait for records; defaults to 1 second.
- `offsetReset` (required): defines the strategy to apply when the consumer starts consuming messages from the queue.
- `maxPolledRecords`: defines the maximum number of records returned in a unique poll batch; defaults to 500.
- `pollDelay` (required): specifies the delay between query executions.
- `monitoring`(optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.

Tips

- Use the `consume` step with a left join to assert data or inject data into a workflow.
- Valid `offsetReset` strategies are:
  - `OffsetResetStrategy.EARLIEST`: starts consuming messages from the beginning of the queue.
  - `OffsetResetStrategy.LATEST`: starts consuming messages when the consumer starts, ignoring all previous messages in the queue.

    `OffsetResetStrategy.LATEST` is the default.
- Refer to the APACHE KAFKA documentation for further parameter and configuration information.

Produce step

The `produce` step within the `kafka` plugin pushes data onto topics of the Kafka broker and forwards the data as input to the next step.

Ancestor

Scenario

Functionality

The `produce` step's input and step context are used to generate values to complete the step. The batch of results, or a single result, is then forwarded to the next step(s).

Example

```
kafka().produce(
   Serdes.ByteArray().serializer(),
   Serdes.String().serializer()
) {
   name = "kafka-producer"
   bootstrap(bootstrap)
   clientName("producer")
   records { _, input ->
       listOf(
           KafkaProducerRecord(
               topic = producerTopic,
               value = "Produced ${input.record.value!!}"
           )
       )
   }
        monitoring {
            events = true
            meters = false
        }
}
```

Notable parameters

- `Serdes.ByteArray().serializer()`: describes how to serialize the record keys to send to `kafka`.
- `Serdes.String().serializer()`: describes how to serialize the record values to send to `kafka`.
- `bootstrap` configures the bootstrap hosts of the Kafka cluster; defaults to `localhost:9092`.
- `clientName` defines the name of the `kafka` client
- `records` (required): generates and configures a list of produced records.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tip

- Refer to the APACHE KAFKA documentation for further parameter and configuration information.

mongodb plugin

Overview

The `mongodb` plugin connects QALIPSIS to MongoDb.

Technology addressed

MongoDB: https://mongodb.com/

Dependency

`io.qalipsis:plugin-mongodb`

Namespace in scenario

`mongodb()`

Client library

MongoDB Reactive Streams Driver: refer to MongoDB Java Reactive Streams.

Supported steps

- `poll`: polls the newest data periodically.
- `save`: saves a batch of documents generated from the input and step context.
- `search`: searches documents and builds a query from the input and step context.

Poll step

The `poll` step within the `mongodb` plugin polls the newest records from a database table, with a delay between each execution.

Ancestor

Scenario

Functionality

The `poll` step is created only once in a scenario; it is then used throughout the scenario as called. The `poll` step within the `mongodb` plugin uses a strategy of “delivered at least once”.

Example

```
mongodb().poll {
  connect {
MongoClients.create("mongodb://localhost:${mongoDbPort}/?streamType=netty")
  }
  search {
    database = "the-db"
```

```
    collection = "moves"
    query = Document().append("action", "IN")
    sort = linkedMapOf("timestamp" to Sorting.ASC)
    tieBreaker = "timestamp"
  }
  pollDelay(Duration.ofSeconds(2))
}
```

Notable parameters

- `connect` (required): configures the connection to MongoDb.
- `query` (required): specifies a document respecting the requirements of the target database.
- `tieBreaker` (required): repeats the first column used for sorting and comparison (`⇐` or `>=`, depending on the ascending or descending order).
- `parameters` (optional): binds values to the clauses of the prepared query.
- `pollDelay` (required): specifies the delay between query executions.
- `monitoring`(optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.

Tips

- Use the `poll` step with a join operator if you need to verify the values saved by your tested system in the database.
- The `query` parameter requires at least one sorting field (`tiebreaker` in the example) to filter the newest results and not fetch the same records repeatedly.
- The latest known value for `tiebreaker` is used to filter out the values already received from the database.
- Refer to the Mongodb documentation for further parameter and configuration information.

Save step

The `save` step within the 'mongodb` plugin persists a batch of documents into MongoDB.

Ancestor

Step

Functionality

The `save` step's input and step context are used to generate the table, the list of columns and their values, which are then combined and forwarded to the database.

Example

```
mongodb().save {
  connect {
    MongoClients.create()
  }
  query {
    database = databaseName
    collection = collectionName
    records = recordSupplier
  }
        monitoring {
          events = false
    meters = true
  }
}
```

Notable parameters

- `connect` (required): configures the connection to MongoDb.
- `query` (required): specifies a document respecting the requirements of the target database.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Use the `save` step after a `collect` step in order to reduce the number of queries executed onto the database and to reduce the load your scenario generates on the test system.
- The `save` step is extremely flexible; database, collection and documents may depend on the context and input value received from the previous step.
- Refer to the Mongodb documentation for further parameter and configuration information.

Search step

The `search` step within the `mongodb` plugin searches records in the database using a search query.

Ancestor

Step

Functionality

The `search` step's input and step context are used to generate a document and the values to complete it; the batch of results is then forwarded to the next step(s).

Example

```
mongodb().search {
        connect {
    MongoClients.create()
```

```
  }
  search {
    database = { _, _ -> "db" }
    collection = { _, _ -> "col" }
    query = { _, _ -> Document() }
    sort = { _, _ -> linkedMapOf("asc" to Sorting.ASC, "desc" to Sorting.DESC) }
  }
  monitoring {
          events = false
    meters = true
  }
}
```

Notable parameters

- `connect` (required): configures the connection to MongoDb.
- `search` (required): defines the search parameters including `database`, `collection` `query` and `sort`.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Use this step after a collect() step in order to reduce the number of queries executed onto the database and to reduce the load your scenario generates on the test system.
- Refer to the Mongodb documentation for further parameter and configuration information.

netty plugin

Overview

The netty plugin supports TCP, UDP, HTTP and MQTT clients.

Technologies addressed

- HTTP/1.1: https://httpwg.org/
- HTTP/2: https://httpwg.org/
- MQTT: https://mqtt.org/
- TCP (Transmission Control Protocol): https://datatracker.ietf.org/doc/html/rfc793
- UDP (User Datagram Protocol): https://datatracker.ietf.org/doc/html/rfc768

Dependency

`io.qalipsis:plugin-netty`

Namespace in scenario

`netty()`

Client library

Netty: refer to the Netty project.

Supported steps

- `HTTP`: executes an HTTP request using `HTTP 1.1` or `HTTP 2.0`.
- `HTTP WITH`: executes an HTTP request reusing the HTTP connection created by a precedent step.
- `CLOSE HTTP`: closes an HTTP connection created by a precedent step.
- `MQTT Publish`: sends data to an MQTT server.
- `MQTT Subscribe`: consumes data from an MQTT server.
- `TCP`: exchanges data with a TCP server.
- `TCP WITH`: exchanges data reusing the TCP connection created by a precedent step.
- `CLOSE TCP`: closes a TCP connection created by a precedent step.
- `UDP`: exchanges data with a UDP endpoint.

HTTP step

The `HTTP` step within the `netty` plugin executes an HTTP request using `HTTP 1.1` or `HTTP 2.0`.

Ancestor

Scenario and Step

Functionality

- The `HTTP` step establishes a connection with the server using either a pool of connections or an individual connection for each minion.
- If an individual connection is used, the connection is maintained until the latest execution for the minion is performed, including the ones from the related `HTTP WITH` step, as in the example.
- When the step is kept open without limit, a `CLOSE TCP` step must be executed to properly close the connections and avoid saturation of the system.

  Be aware that the number of open connections that can be concurrently maintained depends on the configuration of your operating system. If there are issues with maintaining open connections, try adapting the limits of the file descriptors.

Example

```
netty().http {
   request { context, input -> SimpleHttpRequest(HttpMethod.HEAD, "/head") }
   connect {
       url("https://my-website.com:8080/test")
       connectTimeout = Duration.ofSeconds(5)
       inflate()
       charset(Charsets.ISO_8859_1)
       maxContentLength(123)
       followRedirections(15)
       noDelay = false
       keepConnectionAlive = false
       tls {
           disableCertificateVerification = true
       }
       proxy {
           type = HttpProxyType.SOCKS5
           address("my-proxy", 9876)
       }
       followRedirections(5)
   }
   pool {
       size = 143
       checkHealthBeforeUse = true
   }
   monitoring {
       events = true
       meters = true
```

```
    }
}.deserialize(String::class)
```

Notable parameters

- `request`: a factory to create the request to exchange, using the step context and the input as arguments.
- `connect`: configures the connection to the remote endpoint.
  - `version`: version of the HTTP protocol to use: `HTTP_1_1` or `HTTP_2_0`,; defaults to `HTTP_1_1`.
  - `url`: the root URL, the path of the request is appended to it.
  - `connectTimeout`: limit duration to fully establish the connection; defaults to 10 seconds.
  - `readTimeout`: limit duration to receive a response and read it totally after a request; defaults to 10 seconds.
  - `shutdownTimeout`: limit duration to close a connection; defaults to 10 seconds.
  - `sendBufferSize`: the size of the buffer used to prepare the data to send; defaults to 1024 bytes.
  - `receiveBufferSize`: the size of the buffer to receive the data of the response; defaults to 1024 bytes.
  - `nettyChannelOptions`: overwrites the options of the `netty` channel, use with care.
  - `noDelay`: disables the Nagle Algorithm; defaults to true.
  - `keepConnectionAlive`: when set to true, the HTTP header Connection is set to Keep-Alive and the client connection is kept open until `CLOSEHTTP`() is called.
  - `tls`: configuration for SSL/TLS
    - `disableCertificationVerification`: set it to true when the certificate on server should not be verified (for example for a self-signed certificate)
    - `protocols`: supported protocols; defaults to the `netty` default for the related HTTP version.
    - `ciphers`: supported cipher algorithms; defaults to the netty` default for the related HTTP version.
  - `proxy`: configuration to exchange HTTP data via a proxy; defaults to no configuration.
    - `address`: the host and port of the proxy.
    - `type`: type of the proxy to use: `HTTP`, `SOCKS5`; defaults to `HTTP`.
    - `authenticate`: username and optional password to authenticate to the proxy, if it supports and requires it.

- `charset`: character encoding of the HTTP requests,
- `maxContentLength`: maximal expected size of a request, in bytes, defaults to `1048576`, which is the minimum allowed.
- `inflate`: enables the acceptance of compressed response. Compression of the request is not supported.
- `followRedirections`: when the server returns an HTTP status 3xx (redirection) in the response, the client will resend the request to the provided location. Otherwise, the response is kept as is. The argument maxRedirections provides the maximal number of redirections to follow; defaults to 10. By default, the redirections are not followed at all. Note that following a redirections makes the request to be sent there, exactly as it is (same method and content).

- `pool`: configuration of the pool of connections; the `pool` is disabled by default.
  - `size`: number of constant connections maintained in the pool.
  - `checkHealthBeforeUse`: verifies the health of the connection when polled from the `pool` and before it is used.
- `monitoring` (optional): configures the monitoring parameters for the step.
- `deserialize`: specifies the class to use to deserialize the body of the response.

Tip

- Refer to the Netty documentation for further configuration and parameter information.

HTTP WITH step

The `HTTP WITH` step within the `netty` plugin executes an HTTP request reusing the HTTP connection created by a precedent step.

Ancestor

Step

Functionality

- The `HTTP WITH` step is a direct or indirect follower of an `HTTP` step. `HTTP WITH` specifies the name of the step that owns the connection, and reuses a connection from that step as it was created (either from a pool or the connection attached to the minion).
- `HTTP WITH` cannot configure the connection differently than the `HTTP` step it relates to.

Example

```
netty().httpWith("name-of-a-previous-HTTP-step") {
   request { context, input -> SimpleHttpRequest(HttpMethod.HEAD, "/head") }
```

```
    monitoring { all() }
}.deserialize(String::class)
```

Notable parameters

- `netty().httpWith`("name-of-a-previous-HTTP-step"): the name of the `HTTP` step that owns the connection.
  - `request`: a factory to create the request to exchange, using the step context and the input as arguments.
  - `monitoring`(optional): configures the monitoring parameters for the step.
- `deserialize`: specifies the class to use to deserialize the body of the response.

Tip

- Refer to the Netty documentation for further configuration and parameter information.

CLOSE HTTP step

The `CLOSE HTTP` step within the `netty` plugin closes the `HTTP` connection attached to the minion, and created by a direct or indirect ancestor step, when the tail of the minion workflow is received.

It is always a best practice to use a *CLOSE HTTP* step when reusing an `HTTP` connection across several steps without a pool of connections.

Ancestor

Step

Functionality

The `CLOSE HTTP` step is a direct or indirect follower of an `HTTP` step which keeps the connection open. `CLOSE HTTP` simply forwards the input it receives to the next step(s).

Example

```
netty().closeHttp("name-of-a-previous-HTTP-step")
```

Notable parameters

- `closeHttp` (required) specifies the name of an `HTTP` step owning the `HTTP` connection to close.
- `monitoring`(optional): configures the monitoring parameters for the step.

Tip

- Refer to the Netty documentation for further configuration and parameter information.

MQTT publish step

The `MQTT Publish` step within the `netty` plugin sends data to an MQTT server.

Ancestor

To be provided.

Functionality

To be provided.

Example

```
To be provided.
```

Notable parameters

To be provided.

Tip(s)

To be provided.

MQTT subscribe step

- The `MQTT Subscribe Step` within the `netty` plugin consumes data from an MQTT server.

  ### Ancestor

  To be provided.

  ### Functionality

  To be provided.

  ### Example

  ```
  To be provided.
  ```

  ### Notable Parameters

  To be provided.

  ### Tip(s)

  To be provided

TCP step

The `TCP` step within the `netty` plugin is a client for `TCP`.

Ancestor

Scenario and Step

Functionality

The TCP step establishes a connection with the server using either a pool of connections or an individual connection for each minion.

If an individual connection is used, the connection is maintained until the latest execution for the minion is performed, including the ones from the related `TCP WITH` step, as in the example.

When the step is kept open without limit, a `CLOSE TCP` step should be executed to properly close the connections and avoid a saturation of the system.

Be aware that the number of open connections that can be concurrently maintained depends on the configuration of your operating system. If there are issues with maintaining open connections, adapting the limits for file descriptors may help.

Example

```
netty().tcp {
   request { context, input -> input.encodeToByteArray() }
   connect {
        address("remote-server.acme.com", 9000)
        connectTimeout = Duration.ofMinutes(1)
        noDelay = false
        keepConnectionAlive = false
        tls {
            disableCertificateVerification = true
        }
        proxy {
            type = HttpProxyType.SOCKS5
            address("my-proxy", 9876)
        }
    }
    pool {
       size = 1000
       checkHealthBeforeUse = true
    }
    monitoring {
       events = true
       meters = true
   }
}
```

Notable parameters

- `request`: a factory to create the request to exchange, using the step context and the input as arguments; the request is an array of bytes.
- `connect`:configures the connection to the remote endpoint.
  - `address`:hostname or IP address and port of the remote server.
    - `connectTimeout`: limit duration to fully establish the connection; defaults to 10 seconds.
    - `readTimeout`:limit duration to receive and read a response after a request; defaults to 10 seconds.
    - `shutdownTimeout`:limit duration to close a connection; defaults to 10 seconds.
    - `sendBufferSize`: the size of the buffer used to prepare the data to send; defaults to 1024 bytes.
    - `receiveBufferSize`: the size of the buffer to receive the data of the response; defaults to 1024 bytes.
    - `nettyChannelOptions`: overwrites the options of the `netty` channel, use with care.
    - `noDelay`: disables the Nagle Algorithm; defaults to `true`.
    - `tls`: configuration for SSL/TLS
  - `disableCertificationVerification`: set to true if the certificate on the server should not be verified (for example for a self-signed certificate).
  - `protocols`: supported protocols; defaults to the `netty` default.
  - `ciphers`: supported cipher algorithms; defaults to the `netty` default.
    - `proxy`: configuration to exchange data via a proxy; defaults to no configuration.
  - `address`: hostname or IP address and port of the proxy server.
  - `type`: type of the proxy to use: `SOCKS4`, `SOCKS5`; defaults to `SOCKS4`.
  - `authenticate`: username and optional password to authenticate to the proxy, if the proxy supports and requires it.
- `pool`: configuration of the pool of connections; defaults to disabled.
  - `size`: number of constant connections maintained in the pool.
  - `checkHealthBeforeUse`: verifies the health of the connection when polled from the pool and before it is used.

Tip

- Refer to the Netty documentation for further configuration and parameter information.

TCP WITH step

The `TCP WITH step` within the `netty` plugin sends a TCP payload, reusing the connection created by a precedent step.

Ancestor

Step

Functionality

- The `TCP WITH` step is a direct or indirect follower of a `TCP` step. `TCP WITH` specifies the name of the step that owns the connection, and reuses a connection from that step as it was created (either from a pool or the connection attached to the minion).
- `TCP WITH` cannot configure the connection differently than the `TCP` step it relates to.

Example

```
netty().tcpWith("name-of-a-previous-HTTP-step") {
   request { context, input -> input.encodeToByteArray() }
   monitoring { all() }
}
```

Notable parameters

- `request`: a factory to create the request to exchange, using the step context and the input as arguments.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tip

- Refer to the Netty documentation for further configuration and parameter information.

CLOSE TCP step

The `CLOSE TCP` step within the `netty` plugin closes the TCP connection, attached to a minion and created by a direct or indirect ancestor step, when the tail of the minion workflow is received.

It is best practice to use a `CLOSE TCP` step when reusing a `TCP` connection across several steps without a pool of connections.

Ancestor

Step

Functionality

The `CLOSE TCP` step is a direct or indirect follower of an `HTTP` step which keeps the connection open. `CLOSE TCP` simply forwards the input it receives to the next step(s).

Example

```
netty().closeHttp("name-of-a-previous-HTTP-step")
```

Notable parameters

- `closeHttp` (required): specifies the name of the `HTTP` step to close.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tip

- Refer to the Netty documentation for further configuration and parameter information.

UDP step

The `UDP` step within the `netty` plugin is a client for UDP.

It is best practice to use the `UDP` step when reusing a `TCP` connection across several steps without a pool of connections.

Ancestors: Scenario and Step

Functionality

The `UDP` step creates a UDP Datagram packet from the `request` and sends it to the remote endpoint and expects a response.

Example

```
netty().udp {
    request { context, input -> input.encodeToByteArray() }
    connect {
       address("remote-server.acme.com", 9000)
       connectTimeout = Duration.ofMinutes(1)
    }
    pool {
       size = 143
       checkHealthBeforeUse = true
   }
   monitoring {
       events = true
       meters = true
   }
}
```

Notable parameters

- `request`: a factory to create the request to exchange, using the step context and the input as arguments, the request is an array of bytes

- `connect`: configures the connection to the remote endpoint
  - `address`: hostname or IP address and port of the remote endpoint
  - `connectTimeout`: limit duration to fully establish the connection; defaults to 10 seconds.
  - `readTimeout`: limit duration to receive a response and read it totally after a request; defaults to 10 seconds.
  - `shutdownTimeout`: limit duration to close a connection; defaults to 10 seconds.
  - `sendBufferSize`: the size of the buffer used to prepare the data to send; defaults to `1024` bytes.
  - `receiveBufferSize`: the size of the buffer to receive the data of the response; defaults to `1024` bytes.
  - `nettyChannelOptions`: overwrites the options of the `netty` channel, use with care.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tip

- Refer to the Netty documentation for further configuration and parameter information.

r2dbc-jasync plugin

Overview

The `r2dbc-jasync` plugin connects QALIPSIS to MariaDB, MySQL and PostgreSQL.

Technologies addressed

- MariaDB: https://mariadb.org/
- MySQL: https://www.mysql.com
- PostgreSQL: https://www.postgresql.org/

Dependency

`io.qalipsis:plugin-r2dbc-jasync`

Namespace in scenario

`r2dbcJasync()`

Client library

Jasync-sql: refer to jasync-sql

Jasync-sql is an asynchronous database driver for PostgreSQL and MySQL written in kotlin.

Supported steps

- `poll`: polls the newest data periodically.
- `save`: saves a batch of records generated from the input and step context.
- `search`: searches records and builds a query from the input and step context.

Poll step

The `poll` step within the `r2bc-jasync` plugin polls the newest records from the database, respecting a delay between each execution of a select query.

Ancestor

Scenario

Functionality

The `poll` step is created only once in a scenario; it is then used throughout the scenario as called. The `poll` step within the `r2b-jaysync` plugin uses a strategy of “delivered at least once”.

Example

```
r2dbcJasync().poll {
        protocol(Protocol. POSTGRESQL)
        connection {
                 port = 5432
                 username = "dbuser"
                password = "dbpassword"
                database = "thedatabase"
        }

        query("""select username, "timestamp" from buildingentries where action = ? order by
"timestamp"""")
                parameters("ENTERING")
                pollDelay(Duration.ofSeconds(1))
                monitoring {
                        events = false
            meters = true
                }
    }
```

Notable parameters

- `connection` (required): specifies the connection parameters.
- `query` (required): specifies a query respecting the requirements of the target database (MariaDB, MySQL, PostgreSQL,).
- `parameters` (optional): binds values to the clauses of the prepared query.
- `pollDelay` (required): specifies the delay between query executions. `tieBreaker` (required): repeats the first column used for sorting and comparison (⇐ or `>=`, depending on the ascending or descending order).
- `monitoring`(optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.

Tips

- Use the `poll` step with a join operator when you need to verify the values saved by your tested system in the database.
- `query` requires at least one sorting field (known as `tiebreaker` in the example) to filter the newest results and not fetch the same records repeatedly.
- The latest known value for the `tiebreaker` is used to filter out the values already received directly from the database.
- Refer to Configuring and Managing Connections for further parameter and configuration information.

Save step

The `save` step within the `r2bc-jasync` plugin persists a batch of records into an SQL table using a prepared statement.

Ancestor

Step

Functionality

The `save` step's input and step context are used to generate the table and the list of columns and their values, which are then combined and forwarded to the database.

Example

```
r2dbcJasync().save {
        protocol(Protocol. POSTGRESQL)
        connection {
                 port = 5432
                 username = "dbuser"
                password = "dbpassword"
                database = "thedatabase"
        }
        tableName{ ctx, input -> "my_table" }
        columns{ ctx, input -> listOf("firstname", "surname", "registration") }
        values{ ctx, input -> list(
        JasyncSaveRecord("john", "doe", "265edfg2g5"),
        JasyncSaveRecord("alice", "foo", "3763gdw8"),
        JasyncSaveRecord("bob", "bar", "627twh762")
    )}
        monitoring {
                  events = false
        meters = true
    }
}
```

Notable parameters

- `connection` (required): specifies the connection parameters.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- Use the `save` step after a `collect()` step in order to reduce the number of queries executed onto the database and to reduce the load your scenario generates on the test system.

- The `save` step is extremely flexible; table, columns and values might depend on the context and input value received from the previous step
- Refer to Configuring and Managing Connections for further parameter and configuration information.

Search step

The `search` step within the `r2bc-jasync` plugin searches records in the database using a search query.

Ancestor

Step

Functionality

The `search` step's input and step context are used to generate a prepared statement and the values to complete it; the batch of results is then forwarded to the next step(s).

Example

```
r2dbcJasync().search {
        protocol(Protocol. POSTGRESQL)
        connection {
                 port = 5432
                 username = "dbuser"
                password = "dbpassword"
                database = "thedatabase"
        }
        query { ctx, input -> """select username, "timestamp" from buildingentries where action =
? order by "timestamp""" }
        parameters { ctx, input -> listOf("ENTERING") }

        monitoring {
                  events = false
            meters = true
    }
}
```

Notable parameters

- `connection` (required): specifies the connection parameters.
- `query`: generates a prepared statement from the step's context and input.
- `parameters` (optional): binds values to the clauses of the prepared query.
- `monitoring`(optional): configures the monitoring parameters for the step.

Tips

- Use the `search` step after a `collect` step in order to reduce the number of queries executed onto the database and to reduce the load your scenario generates on the test system.
- Refer to Configuring and Managing Connections for further parameter and configuration information.

rabbitmq plugin

Overview

The rabbitmq plugin connects QALIPSIS to the RabbitMQ broker.

Technology addressed

RabbitMQ: https://www.rabbitmq.com/

Dependency

`io.qalipsis:plugin-rabbitmq`

Namespace in scenario

`rabbitmq()`

Client library

RabbitMQ: refer to rabbitMQ

Supported steps

- `consume`: polls data from the `rabbitmq` broker.
- `produce`: pushes data to the rabbitmq broker.

Consume step

The `consume` step within the `rabbitmq` plugin consumes data from queues of the rabbitmq broker and forwards the data to the next step.

This step is generally used in conjunction with `join` to assert data or inject data into a workflow.

This step supports multithreading concurrent consumers using the `concurrency()` property. When the concurrency is set to more than 1, the sequence of the records transmitted to the next step in no longer guaranteed.

To learn more, visit the RabbitMQ website.

Ancestor

Scenario

Functionality

The `consume` step starts a polling loop for consumption of the designated queue. As new records appear, `consume` polls them and sends them to the next step.

Example

```
rabbitmq().
  consume {
    queue("user")
    connection {
      host = hostContainer
      port = portContainer
      password = "defaultpass"
      username = "user"
    }
    concurrency(2)
    prefetchCount (11)
}
```

Notable parameters

- `queue`: defines the name of the queue to consume.
- `connection`: configures the connection to the RabbitMQ broker; defaults to `localhost:5672`
- `concurrency:` defines the number of concurrent channels producing messages to RabbitMQ; defaults to 1.
- `prefetchCount`: defines the prefetch count value; defaults to 10.
- `pollDelay` (required): specifies the delay between query executions.
- `monitoring`(optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.

Tips

- Use the `consume` step with `join` to assert data or inject data into a workflow.
- Learn more about `prefetchCount` by reading the RabbitMQ Prefetch documentation.
- To configure the `consume` step, consult the rabbitMQ documentation.

Produce step

The `produce` step within the `rabbitmq` plugin provides messages to the `rabbitmq` broker using a query; the messages are then forwarded as input to the next step.

Ancestor

Scenario

Functionality

The `produce` step's input and step context are used to generate the values to complete the step; results are then forwarded to the next step(s).

Example

```
rabbitmq().
  produce {
    name = "my-producer-step"
    concurrency(10)
    connection {
      host = "anotherhost"
      port = 5673
      password = "pass"
      username = "test"
      virtualHost = "/guest"
      clientProperties = mapOf("t" to "test")
      }
    records(recordSupplier)
    monitoring {
      events = true
      meters = false
    }
}
```

Notable parameters

- `concurrency` defines the number of concurrent channels producing messages to `rabbitmq`; defaults to `1`.
- `connection`: configures the connection to the `rabbitmq` broker; defaults to `localhost:5672`.
- `records`: configures and generates a list of produced records.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- For more information on configuring the connection to `rabbitmq`, refer to Connecting to RabbitMQ.
- For more information on configuring the `produce` step, refer to the rabbitMQ documentation.

redis-lettuce plugin

The redis-lettuce plugin connects QALIPSIS to Redis.

Technology addressed

Redis: https://redis.io/

Dependency

`io.qalipsis:plugin-redis-lettuce`

Namespace in scenario

`redisLettuce()`

Client library

Lettuce: refer to Lettuce.

Supported steps

- `poll`: polls the newest data periodically.
- `save`: saves a batch of records generated from the input and step context.
- `consume`: consumes data from `redis` streams.
- `produce`: pushes messages to `redis` streams.

Poll step

The `poll` step within the `redis-lettuce` plugin polls the newest records from a `redis` key, with a delay between each execution of polling.

Ancestor

Scenario

Functionality

The `poll` step is created only once in a scenario; it is then used throughout the scenario as called.

- The `KeyOrPattern` to be used in the poll is set to one of the following valid values: `scan`, `sscan`, `hscan`, `zscan`.
- The native `redis-lettuce` command is called using the `KeyOrPattern` provided.

Example

```
.redisLettuce()
  pollHscan {
    name = "poll.hscanacl"
    connection {
      nodes = “localhost:6379”
      database = 0
      redisConnectionType = RedisConnectionType.CLUSTER
      authPassword = "pass"
      authUser = "user"
    }
    KeyOrPattern("hscan-test-acl")
    pollDelay(Duration.ofSeconds(1))
    monitoring {
      events = false
      meters = false
    }
  }
```

Notable parameters

- `Connection` (required): specifies the connection parameters.
- `redisConnectionType` (required): indicates the connection type as `SINGLE`, `CLUSTER` or `SENTINEL`.
- `KeyOrPattern` (required): specifies the key or pattern that will be polled after each `pollDelay`.
- `pollDelay` (required): specifies the delay between query executions.
- `monitoring` (optional): configures the monitoring parameters for the step.
- `broadcast` (optional): specifies the broadcast parameters for the step.

Tips

- Use the `poll` step with a join operator when you need to verify the values saved by your tested system in the database.
- In addition to HSCAN (`polllHscan` in the example), the `poll` step also supports the following `redis` commands: SCAN (`pollScan`), SSCAN (`pollSscan`), and ZSCAN (`pollZscan`).
- If `redisConnectionType` is set to `SENTINEL`, the parameter masterId is necessary. For more information, refer to: Redis discovery using Redis Sentinel.
- Refer to the Redis documentation for further parameter and configuration information.

Save step

The `save` step within the `redis-lettuce` plugin persists a batch of records into `redis`.

Ancestor

Step

Functionality

The `save` step's input and step context are used to generate a valid `redis` command and the values to complete it; the batch of results is then forwarded to the next step(s). The records are saved individually.

Example

```
.redisLettuce().save {
  name = "save"
  connection {
    nodes = “localhost:6379”
    database = 0
    redisConnectionType = RedisConnectionType.SINGLE
    authPassword = "password"
    authUser = "user"
  }
  records { _, _ ->
    listOf(
      ValueRecord(
        key = "key",
        value = "value"
      ),
      HashRecord(
        key = "hash",
        value = mapOf("key" to "value")
      )
    )
  }
  monitoring {
    events = true
    meters = true
  }
}
```

Notable parameters

- `Connection` (required): specifies the connection parameters.
  - `redisConnectionType` (required): indicates the connection type as `SINGLE`, `CLUSTER` or `SENTINEL`.
- `records` (required): generates and configures a list of saved records.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- The `save` step supports the following `redis` commands: SET, SADD, HSET, ZADD. The provided command may depend on the context and input value received from the previous

step.

- If the `redisConnectionType` is set to `SENTINEL`, the parameter `masterId` is necessary. For more information, refer to Redis discovery using Redis Sentinel.
- Refer to the Redis documentation for further parameter and configuration information.

Consume step

The `consume` step within the `redis-lettuce` plugin records data from `redis` streams.

Ancestor

Scenario

Functionality

The `consume` step creates a consumer group in a `redis` stream with the provided `connection`, consumer `group` name, and `concurrency`. If `concurrency` is set to 1, records are transmitted sequentially to the next step.

Example

```
.redisLettuce()
  .streamsConsume {
    name = "consumer"
    connection {
      nodes = “localhost:6379”
      database = 0
      redisConnectionType = RedisConnectionType.CLUSTER
      authPassword = "password"
      authUser = "user"
    }
    concurrency(5)
    group("consumer-test-1")
    streamKey("test")
    offset(“FROM_BEGINNING”)
    monitoring {
      events = false
      meters = false
    }
  }
```

Notable parameters

- `connection` (required): configures the connection to Qalipsis.
- `redisConnectionType` (required): indicates the connection type as `SINGLE`, `CLUSTER` or `SENTINEL`. The default is `SINGLE`.
- `concurrency` (required): configures the level of coroutines consuming the Redis streams.

- `offset` (required): sets the offset value to `FROM_BEGINNING`, `LAST_CONSUMED` or `LATEST` for each topic and consumer group.
- `broadcast` (optional): specifies the broadcast parameters for the step.
- `pollDelay` (required): specifies the delay between query executions.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- If `redisConnectionType` is set to `SENTINEL`, the parameter `masterId` is necessary. For more information, refer to: Redis discovery using Redis Sentinel.
- If `concurrency` is set to more than 1, the records transmitted to the next step may not be in sequence.
- Refer to the Redis documentation for further parameter and configuration information.

Produce step

The `produce` step within the `redis-lettuce` plugin produces records into the Redis streams.

Ancestor

Step

Functionality

The `produce` step produces a batch of records into one or more `redis` stream topics. The record values may be provided by previous step(s).

Example

```
.redisLettuce().streamsProduce {
  name = "producer"
  connection {
    nodes = “localhost:6379”
    database = 0
    redisConnectionType = RedisConnectionType.CLUSTER
    authPassword = "password"
    authUser = "user"
  }
  records { _, input ->
    listOf(
      LettuceStreamsProduceRecord(
        key = "producer.test",
        value = mapOf("test" to input.value.values.first())
        )
      )
  }
  monitoring {
    events = true
    meters = true
```

```
  }
}
```

Notable parameters

- `redisConnectionType` (required): sets the connection type as `SINGLE`, `CLUSTER` or `SENTINEL`.
- `records` (required): generates and configures a list of produced records.
- `monitoring` (optional): configures the monitoring parameters for the step.

Tips

- If `redisConnectionType` is set to `SENTINEL`, the parameter `masterId` is necessary. For more information, refer to: Redis discovery using Redis Sentinel.
- Refer to the Redis documentation for further parameter and configuration information.

Configuring and Monitoring Test Campaigns

Overview

QALIPSIS aims at giving insights into the behavior of tested systems from a performance perspective.

The QALIPSIS user-interdace provides the tools to create, run, monitor and view results of test campaigns from your browser.

In this section, you will find infornation on topics related to the QALIPSIS User interface.

- Logging in to QALIPSIS
- Navigating the QALIPSIS Interface
- Creating and updating a series
- Configuring and running a campaign
- Displaying campaign data

Logging in to QALIPSIS

1. Navigate to the Qualipsis Welcome page (https://alpha.qalipsis.io)
2. Enter your Username and Password.

   Select the tenant to use from the drop down box.
3. Click on 'Submit' to log into QALIPSIS.

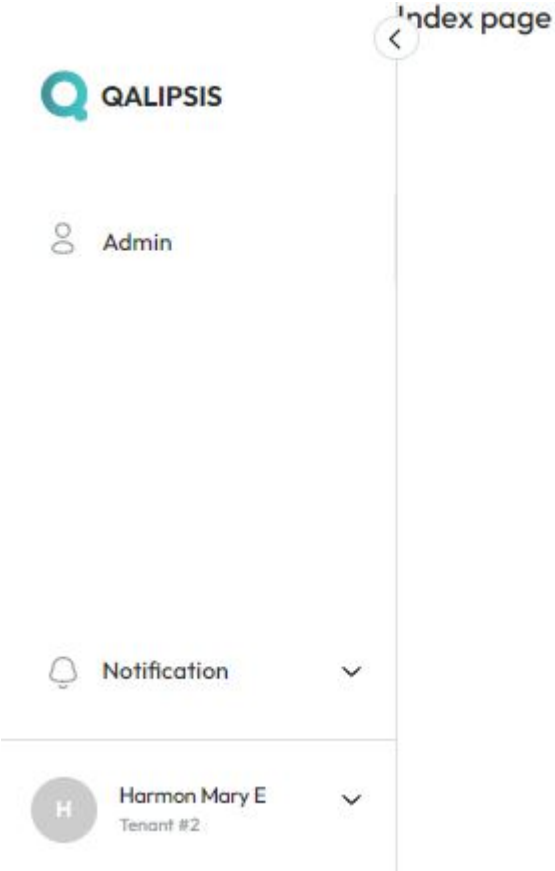
   If a Tenant selection message is not displayed, you will be logged in to QALIPSIS with the tenant you were using in your most recent login.

   You can can select a different tenant at any time once you are logged into the User Interface.

   - Tenant#1 allows access to the *Campaign* and *Series* pages.
   - Tenant#2 allows access to the *Admin* pages.
4. Press the `Continue` button to open the QALIPSIS dashboard.

Navigating the QALIPSIS Interface

When you are logged into QALIPSIS, a left sidebar is displayed and remains displayed throughout your session. The menu near the top of the sidebar depends on the tenant that you are using.



Change to a differnt tenant

1. Click on your name near the bottom of the sidebar to open the *Tenant Modal* window.
2. Select an availablet tenant to use for the current session.
   - With Tenant #1 selected, the *Campaign* and *Series* screens are accessible by clicking on the appropriate link near the top of the sidebar.
   - With Tenant #2 selected, the `Admin` area of the interface is accessible (if you have `Admin` permissions).

View/Update Your profile

From the Tenant #1 sidebar:

1. Click on your name at the bottom of the left sidebar to open the *Tenant Modal* window.
2. Click on the `settings` icon.
3. Click on `My profile` to open the *My Profile* page.
4. Make changes to your profile and click on `Save changes`.

Change your password

From the Tenant #1 sidebar:

1. Click on your name at the bottom of the left sidebar to open the *Tenant Modal* window.
2. Click on the `settings` icon.
3. Click on `My profile` to open the *My Profile* page.
4. Click on `Change password`.
5. Enter your new password in the `New Password` and `Comfirm Password` fields.
6. Click om `Save changes`. to save your new password.

Logout

1. click on your name at the bottom of the left sidebar to open the *Tenant Modal* window.
2. Click on the `settings` icon.
3. Click the `Logout` button. You will be logged out and returned to the *Welcome* screen.

Access Campaign pages

The `Campaign` screen is accessible from the Tenant #2 left sidebar. The `Campaign` screen allows the user to view current campaign data, update a campaign, and create a new campaign. Refer to [Displaying campaign Data] and Configuring and running a campaign for more details.

Access Series pages

The `Series` screen is accessible from the Tenant #2 left sidebar. The Series' screen allows the user view current series information, update or duplicate a series, and create a new series.

Refer to [Viewing series information], [Creating a series], and [Updating a series] for more details.

Access Admin Functions

From the Tenant2 sidebar (if you have `Admin` permissions): . Click on the `Admin` button to display the *Users* screen. . Click on a user's name to open the `Update User` window.

From the `Update User` window, user information can be updated and modified and the Roles (permissions) set.

More options will be available from the `Admin` area of the GUI in the near future.

Creating and updating a series

You can create a series if your profile has write permissions. You can update a series if you are the owner and/or have write permissions.

1. Login as Tenant #1.

2. Click on `Series` in the left sidebar menu.

   The *Series* page opens and displays a table listing currently defined series by name, Type…

3. Click on the ‘+New series’ button near the top right corner of the page.

   The *New series* modal window opens.

4. Enter the required data into the appropriate fields.

5. Click on the `Create` button.

   The following text is displayed in the modal window: '[Series Name] has just been created!'.

6. Close the *New series* modal window to return to the Series page.

   The newly created series is displayed in the table.

You can update a series if you are the owner and/or have write permissions.

1. Login as Tenant #1.

2. Click on `Series` in the left sidebar menu.

   The *Series* page opens and displays a table listing currently defined series by name, Type…

3. Click on athe name of the series you want to update.

   The *New series* modal window opens.

4. Enter the required data into the appropriate fields.

5. Click on the `Create` button.

   The following text is displayed in the modal window: '[Series Name] has just been created!'.

6. Close the *New series* modal window to return to the Series page.

   The newly created series is displayed in the table.

Configuring and running a campaign

1. Click on the `+Campaign` button near the top right of the `Campaign` page.

   A page will open displaying the campaign name, a settings icon, a search field, and a table listing scenarios that are available to run in the campaign.

2. Optionally, rename the campaign by hovering over the `New Campaign` title and selecting the pen icon to edit the name.

3. Select the checkbox to the left of each scenario that you want to include in your campaign.

   Note that each scenarios listed has a default configuration.

   If you want to view and/or update the scenario to better suit your test, click on the `Custom Settings` buttom to the right of the scenario.

‘Changes are saved for the current campaign and doesn’t affect the default Series settings'

4. Optionally, configure the canpaign with custom run-time values.
   a. Click on the arrow next to the ‘Run immediately’ button at the bottom of the page.

      The `custom running` modal window opens.

   b. Select the date you want to start the campaign using the datepicker.

      The date format is [Month dd, yyyy]

   c. Select the time zone for the campaign start time from the ‘Time zone’ dropdown.

      The time zone format is [(UTC+hh:mm) Continent/ City].

   d. If you want the campaign to run more than once, select a frequecy from the [Frequency] drop down.

      The frequecy options are `Daily`, `Weekly`, and `Monthly`.

      `Daily` runs the campaign at the previoulsy selected time each day.

      Weekly runs the campaign on the selected day(s) every week at the previoulsy selected time of day.

      Monthly runs the campaign each selected day(s) of every month at the previously selected time.

Displaying campaign data

1. Login to QALIPSIS as Tenant #1.
2. Click on Campaign in the left sidebar.

   A table listing all existing and planned campaigns is displayed.

   - Click on any column title to sort the table by ascending/descending order.
   - Search for a specific campaign by clicking on the magnifying glass icon near the top right of the screen and entering the name (or partial name) of the campaign.
3. Click on the name of a campaign to display the *Campaign details* page.
   - A report box is displayed at the top of the screen.

     The top line in the report box displays information for the whole campaign; the following lines display information for each scenario within the campaign.

     - Click the `View more` tooltip on the right side of the report box to expand the report box to display all scenarios in the campaign.
     - Click the `More details` tooltip to display the info for the total campaign in a dropdown box.

    - Click on the `Campiagn Summary` dropdown filter to select the scenario information to display in the report box.
  - Below the report box, a row of filter cards is displayed with the individual events and meters that are logged for the campaign.

    Refer to `Step Specifications` for guidance creating filtering steps.

    Each card includes the Name, Type, and display Color.
  - Below the row of filter cards, a chart is displayed with the x-axis displaying time and the y-axis displaying minion count. Select one or more series cards to display the generated data for that series.

Glossary

Term	Definition
Assertion	A step within a scenario that perform cross-checks of data and durations from different sources.
Batch	A collection of values, as opposed to a single value.
Campaign	A single execution of one or a collection of scenarios, starting at a set time and running for a set period.
Cluster	A set of pieces/nodes of the software that are executed in different locations or on different machines.
Dependency Injection	A technique in which an object receives other objects that it depends on, called dependencies. A dependency injection involves four objects each having a different role: * The service object that is injected into a client. * The client object that receives the injected service. * The interface object that defines how the client uses the service. * The injector object that constructs the services and injects them into the client.
Directed Acyclic Graph (DAG)	A subpart of a scenario made up of a single continuous sequence of steps, to be executed in the same location by the same minions. • Root DAG: A DAG starting immediately under a scenario • Frontend DAG: A DAG whose followers execute on a different factory. • Backend DAG: A DAG that continues the execution workflow of a DAG hosted in a different factory. • DAG Underload: A DAG that is executed by the number of minions set on the scenario. • Singleton DAG: A DAG that is executed only once and supports infrastructure or optimization operations.
Directive	A single instruction published by a head to trigger an action in a factory.

Event	A time-stamped snapshot within a step. There are several types of events that can be monitored, including the time to respond, the status of the response, the size of the payload to send, etc.
Feedback	The factory reply to a directive; the feedback provides status of the directive's progression and/or the result of the expected action.
Input	The output of the previous step that is used in the current step's execution.
Load-injected tree	A branch of a scenario that receives load from all of the minions.
Meter/Step Meter	• A meter is a value aggregating step of the performance over a short period of time across all the minions. • A step meter is a unique value representing the performance of a unique execution of a step by a minion (e.g., time to response or volume of data saved, etc.)
Minion	A virtual actor simulating a website user, an IoT device, or another system that interacts with the test system to inject load and verify performance.
Monitoring/Time-series data	A set of records generated by the execution of a campaign that permit the analysis of the performance of the target system and the generation of dashboards.
Node	A software entity of a QALIPSIS cluster. Nodes communicate with each other to exchange data. • Head: Node of a QALIPSIS cluster that coordinates and maintains the state of the cluster, orchestrating the factories and the campaigns. • Factory: Node of a QALIPSIS cluster that executes the scenario to generate load to a target system and verify its behavior.
Operator	A command that allows you create or configure a scenario or a step.
Output	Any data that a step generates, which may be used to verify behavior or generate further load.

Plugin	A technical extension designed to extend the core capabilities of QALIPSIS.
Ramp-up	An operator that defines the pace that the minions are injected into the system. QALIPSIS out of the box ramping operators include: • Timeframe • More • Faster • Regular
Report	• Step report: A consolidated view of the executions of a step. • Campaign report: A consolidated view of the step reports.
Result	The record returned by a step when executed by a minion. The result contains values (e.g., records fetched from a database or a response to a request) and the step meters of the execution. The result is dispatched as output of the step context to the followers.
Runner	A component in charge of triggering and orchestrating the flow of data from one step to the next. The runner is responsible for ensuring that output data from each step is provided as input to the next step as appropriate. Each runner task is assigned to the minion that simulates the user or device executing it.
Run-time parameters	Scenario parameters that are set at run-time or execution of the scenario.
Scenario	Sequence of actions to be executed by minions or to poll data from a remote system.
Selector	The Key/Value pair attached to a step to locate its execution in a factory owning the same pair.
Specification	The configuration of a step or a scenario that describes the actions to perform and the sequence in which they are to be performed.
Step	The central atomic components within each scenario. Steps are the challenges minions must complete in order to continue in the

	scenario. Each step has purpose. There are several step types within QALIPSIS: • Error processing steps that contains information for handling a failed assertion or execution. • Proxy steps that act as a middle man to forward the step execution to a follower, using convenient strategies. Operator steps that act on its data input or output before releasing it to the next step, • Assertion steps that validate incoming or outgoing data.
Step context	A single execution of a single step by a single minion. minion * Execution: A set of fields and values representing the single execution of a single minion on a single step. • Exhausted: After an execution failure, a step context is marked as exhausted, and this state is propagated to its descendants. • Tail: A step context that represents the last one in the execution workflow of a single minion.
Target system	An IT system that QALIPSIS injects load into in order to verify its ability to scale and support a high volume of users or data.

Frequently Asked Questions

- **How do I implement a plugin into a Scenario?**

  Answer text here

- **How do I change the ramping strategy for a scenario?**

  Answer text here

Architecture

Work in progress

Version 0.4.16-SNAPSHOT
Last updated 2023-01-16 16:30:59 -0500